

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное
образовательное учреждение высшего образования
«ПЕРМСКИЙ ГОСУДАРСТВЕННЫЙ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ»

К. В. РЯБИНИН

**ВИРТУАЛЬНАЯ РЕАЛЬНОСТЬ
И МУЛЬТИМЕДИА**
**ПОСТРОЕНИЕ ВИРТУАЛЬНОГО МИРА
СРЕДСТВАМИ OPENGL**

*Допущено методическим советом Пермского государственного
национального исследовательского университета в качестве
учебного пособия для студентов, обучающихся по направлению
подготовки магистров «Прикладная математика
и информатика»*



Пермь 2018

УДК 004.9
ББК 32.973.26-04
Р 98

Рябинин К. В.

Р 98 Виртуальная реальность и мультимедиа. Построение виртуального мира средствами OpenGL: учеб. пособие / К. В. Рябинин; Перм. гос. нац. исслед. ун-т. – Пермь, 2018. – 100 с.: ил.

ISBN 978-5-7944-3051-6

Пособие нацелено на изучение студентами основ работы с трёхмерной графикой в контексте построения интерактивных виртуальных миров, приобретение практических навыков и компетенций разработки приложений виртуальной реальности с учётом современного графического оборудования. Излагаемый теоретический материал в равной степени относится ко всем низкоуровневым стандартам вывода графики; практическая часть приводится на примере открытого интерфейса OpenGL.

Предназначено для студентов, обучающихся по направлению подготовки магистров «Прикладная математика и информатика», изучающих дисциплину «Виртуальная реальность и мультимедиа», а также будет полезно студентам других направлений и специальностей, изучающим основы программного построения изображений средствами ЭВМ и осваивающим современные технологии разработки мультимедийных информационных систем.

Ил. 18. Библиогр. 9 назв.

УДК 004.9
ББК 32.973.26-04

*Печатается по решению редакционно-издательского совета
Пермского государственного национального исследовательского университета*

Рецензенты: к. т. н., доцент каф. информатики и вычислительной техники Перм. гос. гум.-пед. ун-та **И. П. Половина**;
ИПИМ им. М. В. Келдыша РАН (рецензенты – зав. отделом компьютерной графики и вычислительной оптики, д. ф.-м. н. **В. А. Галактионов**, старший научный сотрудник, к. ф.-м. н. **А. Е. Бондарев**)

ISBN 978-5-7944-3051-6

© Пермский государственный национальный
исследовательский университет, 2018
© Рябинин К. В., 2018

Оглавление

Введение	4
1. Моделирование ландшафта	9
1.1. Создание рельефа местности	10
1.2. Окраска ландшафта	12
1.2.1. Текстура и текстурные координаты	13
1.2.2. Выборка цвета из текстуры	16
1.2.3. Мипмэппинг	19
1.2.4. Анизотропная фильтрация	22
1.2.5. Режимы повторения текстуры	24
1.2.6. Работа с текстурой	27
1.3. Освещение ландшафта	31
1.3.1. Модель освещения Фонга	32
1.3.2. Вычисление и трансформация нормалей	37
2. Моделирование неба	45
3. Пространственная ориентация объектов	52
3.1. Ось и угол	54
3.2. Матрица направляющих косинусов	54
3.3. Углы Эйлера	55
3.4. Кватернионы	56
4. Моделирование камеры	64
4.1. Камера от первого лица	65
4.2. Камера от третьего лица	74
4.3. Орбитальная камера	75
5. Загрузка 3D-моделей	79
6. Рельефное текстурирование	83
Заключение	98
Список литературы	99

Введение

На сегодняшний день словосочетанием *виртуальная реальность* (англ. *Virtual Reality*, VR) обычно обозначают созданный техническими средствами мир, передаваемый человеку через его ощущения (чаще всего – через зрение и слух, однако иногда и через осязание, значительно реже – через обоняние и вкус) и обеспечивающий глубокий эффект присутствия.

Слово *виртуальный* часто используется в компьютерных науках (англ. *Computer Science*) в составе различных инженерно-технических терминов (например, «виртуальные функции», «виртуальная память», «виртуальные ресурсы» и т. п.) и, как правило, означает «реализованный программно, симулированный, имитированный с помощью компьютера», т. е. используется как антоним слова «реальный».

Этимология слова «виртуальный» достаточно любопытна. В контексте компьютерных наук оно пришло в русский язык как калька английского слова *virtual* (однако следует отметить, что, например, в физике оно использовалось и до массового распространения компьютеров, например, в составе словосочетания «виртуальные частицы»). Современные англо-русские словари дают несколько переводов прилагательного «virtual», среди которых «действительный» и «мнимый»¹: два противоположных значения, выбор одного из них целиком контекстуален, что говорит о некоторой необычности этого слова. В английский оно пришло из французского «virtuel» – «возможный», а во французский – из латинского «virtus» – «сила», «доблесть», «способность», что, в свою очередь, происходит от латинского «vir» – мужчина. Кроме того, «Virtus» (в русскоязычном варианте – Виртута) в римской мифологии – богиня воинской доблести, спутница бога войны Марса.

Виртуальная реальность включает в себя понятие *виртуального мира* (англ. *Virtual World*) – совокупности объектов (как правило, трёхмерных), размещённых в некотором пространстве, подвер-

¹Применительно к оптике, например, «virtual image» – мнимое изображение.

женных действию специфических законов (например, законов оптики, механики и т. п.) и допускающих какое-либо взаимодействие с пользователем (начиная от простой навигации в пространстве между объектами и заканчивая сложными моделями влияния на них, например, моделями разрушения, пересборки и т. п.). Однако помимо собственно виртуального мира виртуальная реальность предполагает наличие т. н. *погружения* (англ. *Immersion*) – комплекса мер по максимизации эффекта присутствия с целью заставить пользователя почувствовать себя полностью вовлечённым в события внутри виртуального мира. Если «традиционный» виртуальный мир направлен на то, чтобы пользователь идентифицировал себя с некоторым виртуальным персонажем внутри этого мира, то виртуальная реальность стремится полностью стереть грань между виртуальным персонажем и пользователем [1].

Именно в таких патетических формулировках обычно и описывают виртуальную реальность в СМИ, что придало ей в общественном сознании достаточно красочный, но при этом и окружённый массой предрассудков образ. Этот образ подпитывается ещё и художественным творчеством: виртуальная реальность – излюбленный сюжетобразующий элемент в литературе и кино жанра научной (и околонаучной) фантастики.

В медийной среде виртуальная реальность в первую очередь позиционируется как мощный инструмент индустрии развлечений, способный дать пользователю значительно более яркие впечатления, чем традиционные средства потребления контента. В художественных произведениях, сюжеты которых, как правило, базируются на некотором конфликте, виртуальная реальность, напротив, обычно представляется как некая «ловушка» для человека, инструмент подмены реальной жизни компьютерной имитацией (пожалуй, самым ярким, хотя и далеко не первым в истории, примером здесь можно считать кинотрилогию «Матрица», англ. *Matrix Trilogy*).

В научной среде, где теоретические и технические достижения предстают в очищенном от эмоциональной окраски виде, виртуальная реальность рассматривается как перспективный вид человеко-машинного взаимодействия, способный дать пользовате-

лю богатый набор инструментов для решения самых разнообразных задач, не ограничивающихся одними лишь развлечениями, а, как минимум, включающих исследовательскую и образовательную сферы. Однако очевидной является и необходимость совершенствования этих средств с точки зрения эргономики, как и необходимость выработки санитарных правил и норм работы с ними.

С технической точки зрения виртуальная реальность отличается от традиционного виртуального мира следующими особенностями [1]:

1. Естественность 3D-эффектов: структура виртуального мира и навигация в нём делаются максимально натуральными, согласующимися с человеческим опытом, чтобы не вызывать когнитивного диссонанса, способного разрушить «правдивость» создаваемой иллюзии.
2. Камера от первого лица: виртуальный мир пользователь видит глазами виртуального персонажа; как правило, возможность взглянуть на происходящее «со стороны» умышленно заблокирована.
3. Подавление отвлекающих факторов: «переключение» как можно большего числа органов чувств человека на искусственные раздражители и минимизация физической возможности «отвлечься» от наблюдения виртуального мира в пользу наблюдения реального.
4. Обеспечение естественного манипулирования объектами и естественной навигации: отказ от управления виртуальным персонажем при помощи кнопочных интерфейсов в пользу более свойственных человеку движений.

Первые два пункта выполнимы чисто программными средствами. Третий и четвёртый требуют специализированного аппаратного обеспечения для организации человеко-машинного интерфейса. При качественном исполнении они ведут к «изоляции»: возникновению психологического эффекта «исключённости» из реального мира и «помещённости один на один» с виртуальным, а затем к «погружению» – наивысшему проявлению изоляции, когда

человек начинает полностью (или хотя бы частично) ощущать себя частью виртуального мира.

Именно феномен погружения является одновременно и самой привлекательной, и самой пугающей особенностью виртуальной реальности. На сегодняшний день, однако, полное погружение практически невозможно по причине несовершенства аппаратной части, её относительно низкой эргономичности и ориентированности в первую очередь на зрительные образы и восприятие звука. Осязание, обоняние, вкус, чувство равновесия – всё это «остаётся в реальном мире», хотя эксперименты в области создания аппаратуры для стимуляции соответствующих органов чувств ведутся достаточно активно. Существующие санитарные нормы в рекомендательном порядке регламентируют время погружения: на современном оборудовании рекомендуется делать, как минимум, часовые перерывы на каждую 15-минутную сессию погружения. Цель такого регламента: защитить человека от переутомления и негативных побочных эффектов, таких, например, как морская болезнь, которая, к сожалению, на сегодняшний день является неотъемлемым спутником большинства популярных средств погружения.

Наиболее популярными средствами погружения выступают *шлем (очки) виртуальной реальности* (англ. *Head Mounted Display*) в связке с наушниками и, в идеале, специализированными контроллерами – джойстиками или, ещё лучше, *перчатками виртуальной реальности* (англ. *Data Gloves*). Шлем виртуальной реальности обеспечивает передачу изображения (как правило – стереоскопического) прямо в глаза пользователю, «отрезая» его от внешнего реального мира, а также позволяет отслеживать повороты его головы (а в более сложном случае и её положение), чтобы соответствующим образом изменять ракурс наблюдения сцены. Перчатки (или иные средства захвата движения) позволяют детектировать пространственные жесты пользователя и организовать естественную манипуляцию объектами в виртуальном мире.

В более сложном случае этот набор средств пополняется специализированными беговыми дорожками и средствами определения

движений всего человеческого тела, увеличивая тем самым степени свободы пользователя в виртуальном мире.

Программная часть виртуальной реальности – создание соответствующего виртуального мира – базируется на методах и средствах работы с мультимедийным контентом: изображениями (в частности, трёхмерными), анимацией, видео, звуком и текстом, которые объединены в систему при помощи средств интерактивного взаимодействия.

Данное учебное пособие знакомит читателя с самыми базовыми приёмами создания виртуального мира, пригодного для включения в систему виртуальной реальности. Рассматриваются вопросы моделирования реалистичного открытого пространства, навигации по нему и размещения в нём трёхмерных объектов.

Под «открытым пространством» понимается антипод помещения – виртуальный мир, действия в котором происходят «под открытым небом». Такой вид пространств достаточно хорошо подходит для создания различного рода игр и симуляторов, использующих технологии виртуальной реальности.

В качестве стандарта визуализации, на примере которого продемонстрированы практические реализации всех рассмотренных алгоритмов и приёмов, использован OpenGL [2]. Этот стандарт является открытым, поддерживается абсолютным большинством современных видеокарт (включая графические чипы мобильных устройств), а библиотеки с соответствующим ему API доступны практически для любой популярной операционной системы. Нативным языком программирования для стандарта OpenGL является С. При этом существуют биндинги в самые разные языки и даже специальная браузерная версия стандарта – WebGL, основанная на языке JavaScript. Примеры кода, приведённые в данном пособии, написаны на языке C++, так как он зачастую оказывается наиболее удобен и эффективен при разработке графических приложений, но все обращения к API-функциям OpenGL осуществляются в стиле языка С.

1. Моделирование ландшафта

Важной составляющей виртуального мира является *субстрат* (англ. *Terrain*), по которому (или над которым) движется персонаж, управляемый пользователем системы виртуальной реальности. В зависимости от конкретной решаемой задачи (логики устройства создаваемого виртуального мира) подходы к моделированию такого субстрата могут сильно различаться. В самом простом случае субстратом выступает плоскость, изображающая ровную поверхность (например, пол в некотором помещении). В более сложных случаях (например, при моделировании открытых пространств) субстрат представляет собой природный ландшафт, передавая рельеф некоторой местности. В самом сложном случае субстрат динамически изменяется под влиянием каких-либо внешних сил (например, при моделировании поверхности воды).

Поскольку в данном учебном пособии рассматривается вопрос создания виртуального мира на основе открытого пространства, в качестве субстрата предлагается выбрать ландшафт.

Моделирование реалистичного ландшафта предполагает учёт различных природных факторов, таких как эрозия почвы, сейсмическая активность, течение подземных вод и т. п. Однако в этом случае сложность реализации системы виртуальной реальности и её требования к вычислительным ресурсам многократно повышаются. Поэтому очень часто задача построения ландшафта решается в значительно более простой постановке: создать поверхность, *визуально близкую* к природному рельефу. Критерием качества при этом выступает не геологическая точность результирующей модели, а её визуальная привлекательность.

Для обеспечения визуального качества, достаточного для комфортного восприятия ландшафта пользователем, необходимо, как минимум, учесть в создаваемой модели следующее:

1. Рельеф, соответствующий изображаемой местности.
2. Окраска (текстура, англ. *Texture*), соответствующая изображаемой природной зоне.

3. Освещение, совпадающее с окружающими источниками света.
4. Взаимодействие виртуального персонажа с рельефом.

Рассмотрим моделирование ландшафта в соответствии с вышеуказанными пунктами.

1.1. Создание рельефа местности

Один из самых простых и в то же время достаточно гибких способов задания рельефа – *карта высот* (англ. *Height Map*). Это двумерная матрица $m \times n$, каждая ячейка которой выражает высоту ландшафта в соответствующей точке. В качестве такой матрицы чаще всего используют изображения в градациях серого, передающие высоты значениями яркости пикселей.

Алгоритм воссоздания рельефа местности по карте высот состоит из следующих шагов:

1. Получить изображение размера $m \times n$ в градациях серого, играющее роль карты высот (сгенерировать его процедурно или загрузить из файла, возможно, приведя полноцветное изображение к градациям серого).
2. Создать регулярную сетку из $m \times n$ узлов (вершин), лежащих в плоскости XOZ и объединённых в квадраты (реже – гексагоны), состоящие из треугольников. Шаг сетки (расстояние между узлами, т. е. длина стороны квадрата) выбирается исходя из логики приложения; обычно удобно бывает задать его единичным.
3. Сместить каждый узел сетки по оси Y на $c \cdot h$, где c – значение яркости цвета пикселя карты высот, номер которого соответствует номеру обрабатываемого узла, h – масштабный коэффициент, выбираемый в соответствии с логикой приложения.

Результат выполнения этого алгоритма представлен на рис. 1.

Карты высот можно создавать в графических редакторах, что, в свою очередь, позволяет достаточно гибко управлять рельефом изображаемой местности.

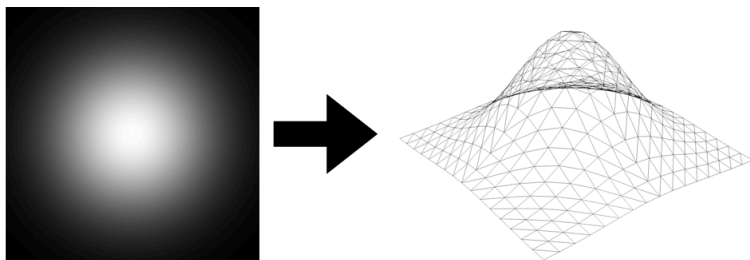


Рис. 1. Построение поверхности ландшафта по карте высот

Для создания сетки (являющейся, по сути, геометрической моделью поверхности ландшафта) удобно использовать индексацию, чтобы, задав порядок обхода узлов при отрисовке, объединить их в треугольники без дублирования данных. При объединении узлов в треугольники важно соблюдать всюду одинаковый порядок их обхода (обычно против часовой стрелки). Причины этого описаны в разделе 1.3.2.

Следует отметить, что подход к построению ландшафта на основе карты высот имеет и свои ограничения: например, с его помощью нельзя симитировать грот, пещеру или арку. Для создания подобных элементов необходимо либо загружать их в виде отдельных 3D-моделей из файлов (см. главу 5), либо комбинировать несколько поверхностей, построенных по разным картам высот.



Задание. Загрузить из файла изображение в градациях серого, содержащее некоторый узор. На основе этого изображения создать массив вершин и массив индексов геометрической модели поверхности ландшафта в соответствии с приведённым выше алгоритмом.

Вывести полученную модель на экран средствами OpenGL.

Подсказка 1. Изображение, которое можно использовать в качестве карты высот, без труда найдётся в сети Интернет. Однако для отладки имеет смысл создать в любом графическом редакторе некоторый простой узор, например, размытый белый круг в центре чёрного

квадратного поля, как на рис. 1. В качестве формата изображения проще всего использовать BMP или TGA (без сжатия), так как их можно легко загрузить без использования сторонних библиотек.

Подсказка 2. При выводе поверхности на экран следует правильно подобрать матрицу преобразования. Сетку удобнее всего создавать в положительном квадранте плоскости XOZ размером $m \times n$, где m и n – ширина и высота карты высот соответственно. Таким образом, чтобы перенести полученный ландшафт в NDC (англ. *Normalized Device Coordinates* – нормированные координаты устройства, в случае OpenGL – куб с центром в нуле и стороной длины 2), необходимо сместить его на $-m/2$ и $-n/2$ по осям X и Z соответственно, масштабировать по X и Z на $1/\max\{m, n\}$ и затем, для большей наглядности, повернуть вокруг оси X на небольшой угол.

Подсказка 3. В целях отладки вершинам имеет смысл добавить 3 дополнительных атрибута со случайными числами из отрезка $[0; 1]$, которые затем передать во фрагментный шейдер, где интерпретировать как цвет. Это, хотя и даст весьма сюрреалистичный результат, позволит визуально оценить корректность рельефа в условиях отсутствия правильной окраски: в учебных целях рекомендуется идти строго последовательно и после каждой новой модификации кода проверять работоспособность приложения, чтобы при возникновении проблем иметь лишь небольшой участок программы для поиска ошибок.

Замечание. Поскольку каждому пикселю карты высот соответствует один узел сетки, детализация ландшафта напрямую зависит от размера изображения. Для получения достаточно гладких поверхностей рекомендуется использовать карты высот размером не меньше чем 128×128 пикселей.

1.2. Окраска ландшафта

Визуальное качество и узнаваемость ландшафта во многом зависят от его окраски. Используя одну и ту же геометрическую модель поверхности, но изменяя её визуальные свойства, можно имитиро-

вать различные природные зоны, разное время года, разные составы почвы и горной породы и т. д.

1.2.1. Текстура и текстурные координаты

В основе окраски ландшафта, как правило, лежит механизм *наложения двумерных текстур* (англ. *2D Texture Mapping*). Его основная идея состоит в том, чтобы по некоторым правилам спроецировать (наложить) на трёхмерную поверхность одно или несколько двумерных изображений (текстур), содержащих требуемые узоры. Такие изображения обычно получают путём фотографирования реальных природных объектов (например, камней, грунта, травы и т. п.) и последующей обработки этих фотографий в графических редакторах с целью адаптирования к задаче наложения на требуемые поверхности (например, производится «склейка» фрагментов различных фотографий, устранение видимых стыковочных швов, цветовая и тоновая коррекция и т. п.).

Для задания правила наложения изображений на трёхмерную поверхность используются т. н. *текстурные координаты* (англ. *Texture Coordinates*) – пары чисел, заданные в качестве атрибутов вершин полигональной поверхности, определяющие проекции этих вершин на изображение.

Система текстурных координат декларируется на уровне графического стандарта. На рис. 2 представлена система текстурных координат, используемая в стандарте OpenGL. Оси в ней называются S и T , чтобы не путать текстурные координаты с пространственными. Следует отметить, что в литературе и различном программном обеспечении также часто встречаются обозначения U и V для соответствующих осей, процесс подготовки текстурных координат (задание конкретных значений этих координат каждой вершине поверхности) иногда называют *созданием UV-развёртки* (англ. *UV-unwrapping*), а процесс наложения текстур – *UV-мэппингом* (англ. *UV-mapping*).

Как можно заметить, текстурные координаты меняются в отрезке $[0; 1]$ вне зависимости от соотношения сторон изображения, что делает механизм наложения текстур унифицированным.

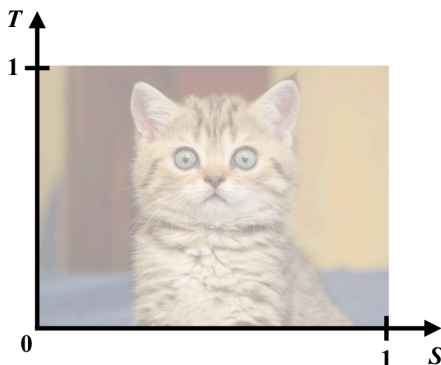


Рис. 2. Система текстурных координат, декларированная стандартом OpenGL

В том случае, если у каждой вершины трёхмерной поверхности заданы (или могут быть вычислены) текстурные координаты, становится возможным определить положение каждого треугольника этой поверхности на текстурном изображении и соответственно цвет каждой внутренней точки этого треугольника, как показано на рис. 3.

Технически раскраска трёхмерных поверхностей осуществляется следующим образом. На этапе преобразования вершин (в ходе работы вершинного шейдера) текстурные координаты либо извлекаются из множества атрибутов обрабатываемой вершины, либо вычисляются для этой вершины в соответствии с логикой вершинного шейдера и, в обоих случаях, передаются дальше по графическому конвейеру. На этапе преобразования фрагментов, полученных в результате растеризации соответствующего треугольника, фрагментный шейдер принимает интерполированные значения текстурных координат и осуществляет *выборку цвета* (англ. *Color Sampling*) из соответствующей текстуры по этим координатам. Полученный цвет используется для определения финального цвета фрагмента (напрямую либо косвенно, в зависимости от логики фрагментного шейдера).

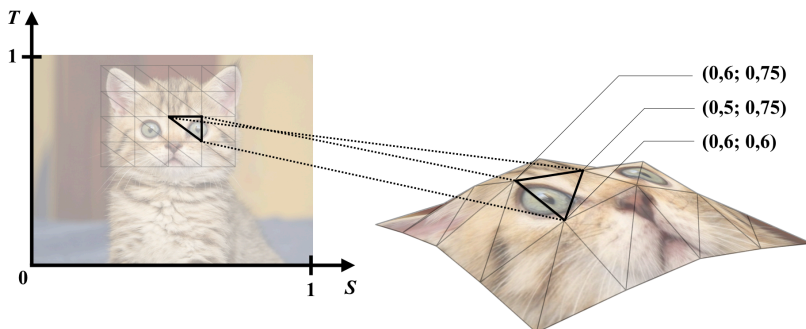


Рис. 3. Наложение двумерной текстуры на трёхмерную поверхность

Наложение текстур имеет на современном оборудовании аппаратную поддержку. При использовании стандарта OpenGL работа с текстурой состоит из следующих этапов:

1. Создание текстурного объекта. Для этого используется функция `glGenTextures`, резервирующая в системной таблице заданное количество записей для хранения текстур. Текстурный объект представляет собой совокупность данных текстурного изображения и сопутствующих настроек, определяющих параметры процесса выборки цвета. Объекты доступны программисту по их беззнаковым целочисленным идентификаторам, при этом идентификатор 0 означает невалидную текстуру.
2. Активация текстурного объекта. Для этого применяется функция `glBindTexture`, которая делает текстуру с заданным идентификатором активной и присваивает ей некоторый тип. Стандарт OpenGL поддерживает различные типы текстур: двумерные (самые распространённые; для них используется константа `GL_TEXTURE_2D`), кубические (будут рассмотрены в разделе 2) и др.
3. Настройка текстурного объекта. Для этого используется серия вызовов функции `glTexParameter`, принимающая 3 параметра-константы: тип настраиваемой текстуры, настраиваемый параметр и устанавливаемое для него значение.

4. Загрузка данных текстуры в видеопамять. Для этого используется функция `glTexImage2D`, которая принимает линейный массив байтов, кодирующих изображение в некотором формате *без сжатия*. Получение этого массива – целиком задача программиста; OpenGL не специфицирует никаких методов загрузки растровых изображений из файлов.
5. Связывание текстуры с графическим конвейером так, чтобы она оказалась доступной в шейдерах, где из неё производится выборка цвета. Для этого на стороне основного приложения используются функции `glActiveTexture` и `glUniform1i`, а на стороне шейдера – функция `texture`. Этот процесс подробно рассмотрен ниже.
6. Удаление неиспользуемого более текстурного объекта. Для этого применяется функция `glDeleteTextures`.

Читателю рекомендуется самостоятельно, используя документацию API OpenGL, ознакомиться с сигнатурами и назначением каждой из вышеупомянутых функций.

Этапы 1–4 и 6, как правило, выполняются однократно (создание, инициализация и удаление текстурного объекта), тогда как этап 5 в общем случае выполняется заново при отрисовке каждого нового кадра.

1.2.2. Выборка цвета из текстуры

Для того чтобы осуществить выборку цвета (*семплирование*, англ. *Sampling*), необходимо вычислить координаты нужного *текселя* (англ. *Texel* – *Texture Element*, элемент текстуры, аналог пикселя применительно к текстуре). Такое вычисление производится путём умножения соответствующих текстурных координат на длину и ширину фактической текстуры. Однако результат этого умножения далеко не всегда выражен целыми числами, т. е. полученные координаты часто не соответствуют адресу какой-то конкретной ячейки текстурного растра, а указывают в пространство между центрами четырёх соседних ячеек.

Самым простым способом выборки цвета по дробным координатам ячейки является их округление. Этот способ носит название

«алгоритм взятия ближайшего соседа» (англ. *Nearest Neighbor*). Для активации этого способа служит константа `GL_NEAREST` для третьего аргумента функции `glTexParameteri`. Однако в этом случае результирующая трёхмерная поверхность не всегда обладает хорошим визуальным качеством: часто на ней оказываются видны артефакты дискретной выборки в виде чётко выраженных ступенчатых границ между разноцветными зонами (см. рис. 4 (слева)).

Чтобы избежать этой проблемы, при выборке используют т. н. фильтрацию (англ. *Filtering*), вычисляя итоговый цвет на основе нескольких соседних ячеек. Часто используется *билинейная фильтрация* (англ. *Bilinear Filtering* – фильтрация на основе билинейной интерполяции) четырёх соседних ячеек, дающая достаточно хороший результат (см. рис. 4 (справа)) при сравнительно небольших вычислительных затратах. Для активации этого способа служит константа `GL_LINEAR` для третьего аргумента функции `glTexParameteri`. Билинейная фильтрация на современном оборудовании имеет аппаратную поддержку.

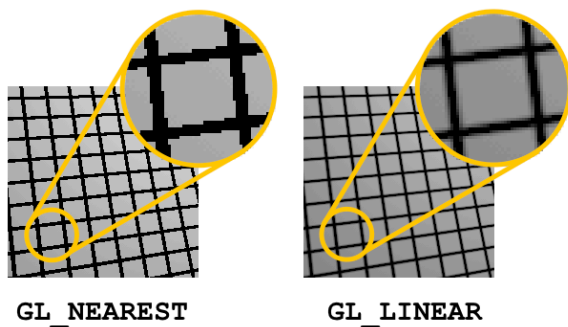


Рис. 4. Пример наложения текстуры с использованием округления координат (слева) и билинейной интерполяции (справа) при выборке цвета

Следующая проблема, которая возникает при работе с текстурами, – несоответствие размера текстуры и размера проекции поверхности, на которую эта текстура накладывается.

Если текстура небольшого размера, например 32×32 текселя, накладывается на поверхность объекта, проекция которого занима-

ет значительную часть площади экрана, например 512×512 пикселей, то каждый тексель будет соответствовать не одному, а целой группе соседних пикселей; в приведённом примере – группе размером 16×16 . Если при этом выборка цвета осуществляется на основе алгоритма взятия ближайшего соседа, результирующее изображение будет выглядеть мозаичным (состоять из чётко очерченных квадратов). Если же используется билинейная фильтрация, изображение будет казаться сильно размытым. Данная ситуация продемонстрирована на рис. 5. Для определения способа выборки цвета, действующего при необходимости увеличить изображение, используется константа `GL_TEXTURE_MAG_FILTER`, передаваемая вторым параметром в функцию `glTexParameterf`.



Рис. 5. Результат наложения текстуры размером 32×32 текселя на квадраты указанного размера с указанным режимом выборки цвета (для наглядности площади пикселей первого изображения увеличены)

В случае, когда текстура большого размера, например 512×512 , накладывается на поверхность объекта, проекция которого на экране очень мала по площади, например 32×32 пикселя, имеет место обратная ситуация: в окраске соответствующего изображения на экране вне зависимости от используемого алгоритма выборки цвета участвуют тексели, расположенные в текстуре далеко друг от друга, в приведённом примере – на расстоянии в среднем в 16 текселей. В результате узор текстуры на поверхности отображаемого объекта нарушается и выглядит рябью. Данная ситуация продемонстрирована на рис. 6. Для определения способа выборки цвета, действующего

при необходимости уменьшить изображение, используется константа `GL_TEXTURE_MIN_FILTER`, передаваемая вторым параметром в функцию `glTexParameterf`.



Рис. 6. Результат наложения текстуры размером 512×512 текселей на квадраты указанного размера с указанным режимом выборки цвета (для наглядности площади пикселей второго и третьего изображений увеличены)

1.2.3. Мипмэппинг

Высокое качество итогового изображения удаётся получить лишь в том случае, когда площадь текстуры близка к площади проекции текстурируемого объекта. Для обеспечения такой близости в условиях, когда ракурс визуализации сцены динамически изменяется (например, у пользователя есть возможность приближать и удалять различные объекты, в результате чего из-за перспективных искажений изменяются площади их проекций), используется технология *мипмэппинга* (англ. *MIP-mapping* – MIP-текстурирование, от лат. *Multum In Parvo* – многое в едином).

Мипмэппинг предполагает хранение в видеопамяти т. н. MIP-пирамиды (англ. *MIP-Pyramid*) – множества версий одной и той же текстуры в разных масштабах с возможностью динамически подобрать нужную версию исходя из площади текстурируемого полигона. Как правило, MIP-пирамиду получают путём уменьшения исходного текстурного изображения до размеров, отличающихся друг от друга в фиксированное число раз. Обычно эти размеры выражают-

ся степенями двойки. Каждую конкретную версию текстуры в МІР-пирамиде называют *МІР-уровнем* (англ. *MIP-Level*). МІР-пирамида продемонстрирована на рис. 7.

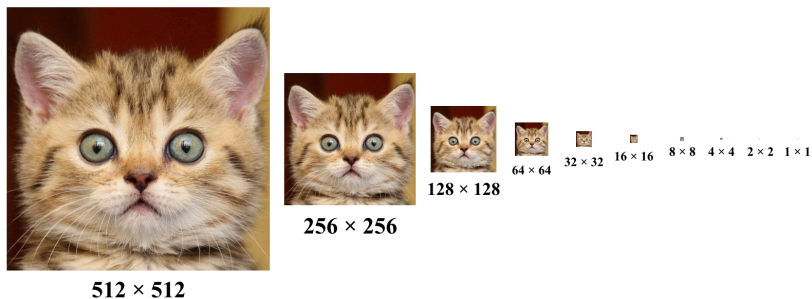


Рис. 7. МІР-пирамида текстуры

Создание МІР-уровней текстуры на основе исходного изображения происходит однократно (как правило, на этапе загрузки графических ресурсов), поэтому при масштабировании может быть использован алгоритм более высокой временной сложности, чем билинейная фильтрация на основе 4 соседних текселей (применяемая при выборке цвета). Это, в свою очередь, обеспечивает более высокое качество масштабированных изображений на каждом МІР-уровне и нивелирует проблему появления ряби на поверхности объектов, площадь проекции которых значительно меньше площади исходного изображения, лежащего в основе МІР-пирамиды соответствующей текстуры.

Выбор нужного МІР-уровня для текстурирования конкретного полигона осуществляется автоматически (на современном графическом оборудовании мипмэппинг имеет аппаратную поддержку). В основе этого выбора лежит идея сопоставления разрешения проекции полигона в заданной точке с уровнями МІР-пирамиды. Алгоритм определения разрешения, как и правило сопоставления, в различных графических стандартах могут различаться. В самом простом случае под разрешением может пониматься площадь, а сопоставление сводится к поиску МІР-уровня, площадь которого ближе всего к пло-

щади проекции текстурируемого полигона. Однако такой подход не обеспечивает требуемого визуального качества, поэтому на практике используются более сложные алгоритмы. Так, например, в стандарте OpenGL разрешение проекции полигона в заданной точке определяется на основе частных производных функции изменения текстурных координат вдоль этой проекции, а сопоставление с MIP-уровнем производится на основе логарифмической связи порядкового номера MIP-уровня с вычисленным разрешением проекции полигона [3, раздел 3.9.11].

Активация мипмэппинга осуществляется путём загрузки подготовленных MIP-уровней в видеопамять, а также указания специального способа выборки цвета для случая уменьшения текстуры (`GL_TEXTURE_MIN_FILTER`).

Подготовка MIP-пирамиды может быть осуществлена как вручную (для этого в функции `glTexImage2D` предусмотрен специальный параметр, отвечающий за порядковый номер загружаемого MIP-уровня), так и автоматически (для чего в OpenGL предусмотрена специальная функция `glGenerateMipmap`, создающая MIP-пирамиду на основе единственного изображения, трактуемого как MIP-уровень максимального размера).

В OpenGL поддерживаются следующие способы выборки цвета при мипмэппинге:

1. `GL_NEAREST_MIPMAP_NEAREST` – выборка цвета производится по алгоритму ближайшего соседа из ближайшего MIP-уровня.
2. `GL_LINEAR_MIPMAP_NEAREST` – выборка цвета производится путём билинейной интерполяции из ближайшего MIP-уровня.
3. `GL_NEAREST_MIPMAP_LINEAR` – выборка цвета производится по алгоритму ближайшего соседа из двух ближайших MIP-уровней; полученные значения линейно интерполируются.
4. `GL_LINEAR_MIPMAP_LINEAR` – выборка цвета производится путём билинейной интерполяции из двух ближайших MIP-уровней; полученные значения линейно интерполируются.

Четвёртый вариант, как правило, обеспечивает самое высокое качество, хотя и является самым вычислительно-сложным. Результат работы мипмэппинга продемонстрирован на рис. 8.

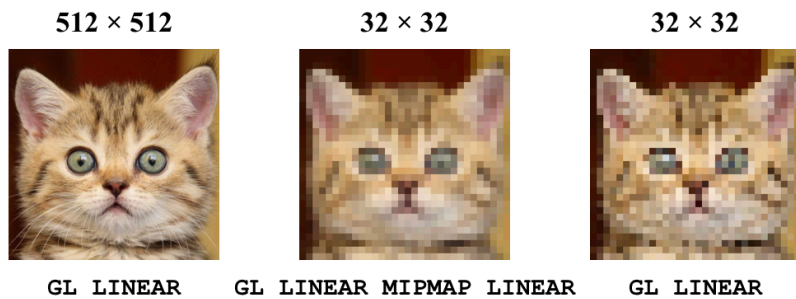


Рис. 8. Результат наложения текстуры размером 512×512 текселей на квадраты указанного размера с указанным режимом выборки цвета (для наглядности площади пикселей второго и третьего изображений увеличены)

Следует обратить внимание, что мипмэппинг действует только на уменьшение текстуры. Увеличение (`GL_TEXTURE_MAG_FILTER`) осуществляется по одному из ранее описанных алгоритмов, применяемых к MIP-уровню максимального размера.

Из-за необходимости хранить несколько MIP-уровней мипмэппинг требует больше памяти, чем обычная текстура. Например, в OpenGL, где площади соседних MIP-уровней отличаются в 4 раза (размеры суть степени двойки), расход памяти увеличивается на $1/3$. В связи с этим, как правило, мипмэппинг используется не для всех объектов сцены, а лишь для наиболее значимых из них. Для создания визуально привлекательного ландшафта мипмэппинг рекомендуется.

1.2.4. Анизотропная фильтрация

Рассмотренные ранее способы выборки цвета из текстуры на основе билинейной интерполяции предполагают, что семплирование осуществляется по одинаковой схеме во всех направлениях

на текстурном изображении, т. е. является *изотропным* (англ. *Isotropic*). Такой подход способен обеспечить высокое качество результирующего изображения главным образом в тех случаях, когда поверхность ориентирована параллельно плоскости экрана. В ситуациях, когда двугранный угол между плоскостью экрана и касательной плоскостью к поверхности увеличивается, стремясь к прямому (например, при визуализации стен и пола в некотором помещении либо же ровных участков ландшафта открытого пространства), точность определения соответствия текселей фрагментам поверхности посредством изотропной выборки ощутимо сокращается. С точки зрения визуального качества, снижение точности локализации текстелей приводит к возникновению «мутных пятен» на текстурном узоре – областей, где произошло нежелательное усреднение цветов.

Для решения этой проблемы используется *анизотропная фильтрация* (англ. *Anisotropic Filtering*) – выборка цвета, схема которой зависит от направления на текстурном изображении. Существует несколько подходов к реализации анизотропной фильтрации, но все они преследуют идею компенсации искажений, возникающих на поверхности вследствие её наклона относительно плоскости экрана. Так, например, зона локализации текселей может иметь не квадратную форму (как в случае билинейной фильтрации), а трапециевидальную, вытянутую вдоль направления обзора, и включать не 4 соседних текселя, а большее их число.

Анизотропная фильтрация на данный момент (2018 г.) не входит в ядро стандарта OpenGL [4], однако поддерживается абсолютным большинством графического оборудования и доступна в виде расширения `EXT_texture_filter_anisotropic` [5].

Для активации этого расширения служит вещественный параметр `GL_TEXTURE_MAX_ANISOTROPY_EXT` настройки текстурного объекта (его значение может быть изменено функцией `glTexParameterf`). Конкретный смысл этого параметра зависит от реализации расширения, однако стандартом оговорено, что он кодирует максимальную величину анизотропии и для включения анизотропной фильтрации должен быть больше 1.

Сравнение изотропной и анизотропной фильтрации представлено на рис. 9.

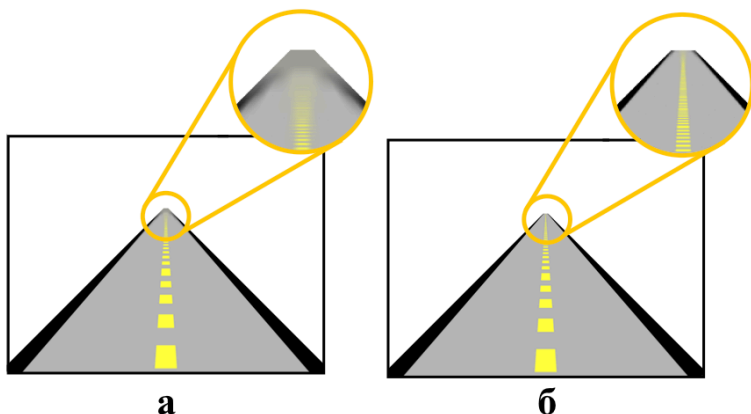


Рис. 9. Результат наложения текстуры при использовании мипмэппинга с изотропной (а) и анизотропной (б) фильтрацией

Как более сложный способ выборки цвета, анизотропная фильтрация требует больше вычислений, потому на практике, как и мипмэппинг, используется только для текстур ключевых объектов. Ещё одним критерием выбора фильтрации служит протяжённость объектов в пространстве и типичная ориентация относительно камеры: чем больше их протяжённость и наклон, тем выше вероятность возникновения на их поверхности артефактов семплирования при изотропной фильтрации и соответственно тем предпочтительнее использование анизотропной. При визуализации ландшафта анизотропная фильтрация, как и мипмэппинг, рекомендована.

1.2.5. Режимы повторения текстуры

Как правило, ландшафты, используемые в качестве субстрата в виртуальных мирах, имеют большую площадь. При этом максимальный размер текстуры на современном оборудовании обычно составляет 4096×4096 пикселей. Чтобы обеспечить высокую детализацию окраски ландшафта, для разных его частей используют раз-

ные текстуры. В частности такой подход позволяет внести в ландшафт разнообразие представленных на нём природных элементов: например, можно скомбинировать текстуры почвы, поросшей травой, каменистого грунта и скал, чтобы симитировать реалистично выглядящие холмы с обнажениями горной породы.

При этом нужно задавать некоторое правило комбинирования. Ландшафт может быть разбит на условные зоны, для каждой из которых будет свой набор текстур. Такие зоны называются *тайлами* (англ. *Tiles*). Цвет каждого фрагмента каждого тайла может вычисляться на основе средневзвешенного значения цветов из всех текстур, связанных с этим тайлом. Весовые коэффициенты могут определяться, например, исходя из высоты фрагмента над условным «уровнем моря».

Площадь тайлов выбирают таким образом, чтобы текстуры поддерживаемого размера обеспечивали надлежащее визуальное качество результата рендеринга с точки зрения чёткости и детализации картинки. Для сокращения числа наборов текстур, требуемых для покрытия всей поверхности ландшафта, часто переиспользуют одни и те же наборы между различными тайлами.

Переиспользование может быть организовано при помощи задания повторяющихся текстурных координат у различных тайлов. Однако более удобным оказывается подход на основе настройки *режимов повторения* (англ. *Wrap Modes*) текстуры. Эти режимы определяют, как будут интерпретироваться координаты за пределами отрезка $[0; 1]$. OpenGL предоставляет 4 различных режима:

1. `GL_REPEAT` – для выборки цвета у заданных текстурных координат, взятых по модулю, берётся дробная часть.
2. `GL_MIRRORED_REPEAT` – для каждой координаты a , взятой по модулю, определяются её целая a_z и дробная a_r части. Затем, если a_z – чётное число, для выборки цвета используется a_r , если же нечётное – используется $1 - a_r$.
3. `GL_CLAMP_TO_EDGE` – для каждой координаты a , если $a < 0$ для выборки используется 0, если $a > 1$ для выборки используется 1.

4. `GL_CLAMP_TO_BORDER` – выборка цвета из текстуры осуществляется только для координат из отрезка $[0; 1]$, для всех остальных значений функция выборки возвращает чёрный цвет (причём, этот цвет не может быть переопределён).

Эти режимы могут быть установлены активному текстурному объекту при помощи функции `glTexParameterf` отдельно для координат по оси S (для чего служит константа `GL_TEXTURE_WRAP_S`, передаваемая вторым параметром этой функции) и по оси T (для чего служит константа `GL_TEXTURE_WRAP_T`).

Результат использования различных режимов при наложении текстуры на плоскость, состоящую из 9 тайлов, показан на рис. 10.

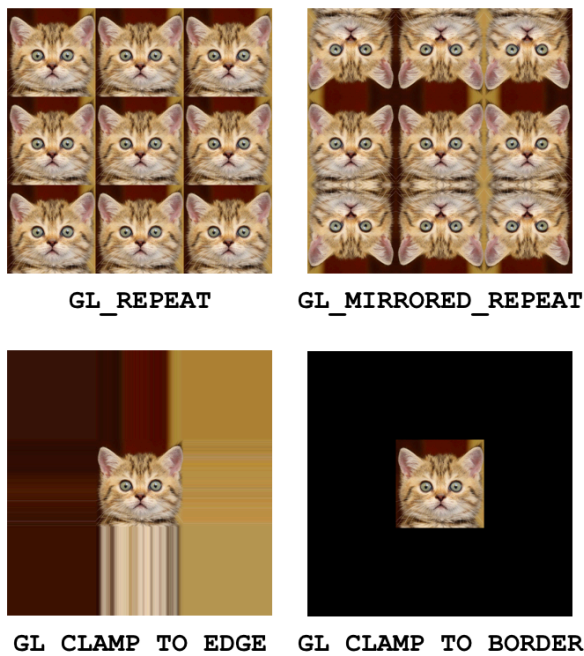


Рис. 10. Различные режимы повторений текстуры

Текстуры, которые могут стыковаться сами с собой без видимых границ, называют *бесшовными* (англ. *Seamless*). Обычно

бесшовные текстуры используют для создания каких-либо архитектурных орнаментов (имеющих в основе регулярный повторяющийся узор) либо же природных поверхностей (почвы, камня, коры деревьев и т. п., имеющих в основе нерегулярный, но самоподобный узор). Для создания бесшовных текстур из обычных фотографий существуют специальные фильтры в составе графических редакторов.

Бесшовные текстуры, в частности, хорошо подходят для наложения на поверхность ландшафта, обеспечивая совместно с режимом повторения `GL_REPEAT` или `GL_MIRRORED_REPEAT` достаточно качественный результат визуализации. Следует отметить, что чаще всего режим `GL_MIRRORED_REPEAT` оказывается более предпочтительным: зеркальное отражение каждого второго тайла даёт эффект большего разнообразия узора и маскирует тайловую природу ландшафта.

1.2.6. Работа с текстурой

Пример исходного кода для создания и настройки текстуры (с учётом использования режима повторения с зеркалированием, автоматической генерации MIP-пирамиды и включения анизотропной фильтрации) приведён в листинге 1. Этот код, в частности, подойдёт для текстуры ландшафта.

Листинг 1. Создание и настройка текстуры

```
1 GLuint handler;
2 glGenTextures(1, &handler);
3 glBindTexture(GL_TEXTURE_2D, handler);
4 glTexParameteri(GL_TEXTURE_2D,
5                 GL_TEXTURE_MAG_FILTER,
6                 GL_LINEAR);
7 glTexParameteri(GL_TEXTURE_2D,
8                 GL_TEXTURE_MIN_FILTER,
9                 GL_LINEAR_MIPMAP_LINEAR);
10 glTexParameteri(GL_TEXTURE_2D,
11                 GL_TEXTURE_WRAP_S,
```

```

12             GL_MIRRORED_REPEAT);
13 glTexParameteri(GL_TEXTURE_2D,
14                 GL_TEXTURE_WRAP_T,
15                 GL_MIRRORED_REPEAT);
16 if (glIsExtSupported(
17     "GL_EXT_texture_filter_anisotropic")) {
18     GLfloat fLargest;
19     glGetFloatv(GL_MAX_TEXTURE_MAX_ANISOTROPY_EXT,
20                 &fLargest);
21     glTexParameterf(GL_TEXTURE_2D,
22                     GL_TEXTURE_MAX_ANISOTROPY_EXT,
23                     fLargest);
24 }
25 glTexImage2D(GL_TEXTURE_2D, 0, internalFormat,
26              width, height, 0,
27              externalFormat, type, bytes);
28 glGenerateMipmap(GL_TEXTURE_2D);

```

Как уже отмечалось выше, действия по созданию, настройке и загрузке текстуры выполняются однократно. В переменную `handler` будет записан идентификатор текстуры, который затем должен быть использован для последующей активации текстуры перед отрисовкой, а также для удаления текстуры, когда она перестанет быть нужна. В связи с этим значение этой переменной нужно сохранить в некотором доступном по требованию месте (часто при создании объектно-ориентированной обёртки над библиотекой стандарта OpenGL текстуру оформляют в виде отдельного класса, полем которого является её идентификатор).

В строке 2 происходит резервирование текстурного объекта, затем, в строке 3, – его активация, после чего все функции изменения состояния начинают действовать именно на данный текстурный объект. Подход на основе активации объекта, а не на основе передачи идентификатора на него в каждый модифицирующий метод, является типичным для OpenGL. Это связано с оптимизацией доступа к памяти видеокарты (минимизацией числа изменений состояния графического конвейера).

В строках 4–9 настраивается базовый способ фильтрации текстуры, в строках 10–15 – режим повторения.

Блок из строк 16–24 отвечает за активацию анизотропной фильтрации (с установкой максимального уровня анизотропии) в том случае, если данное расширение доступно на текущем оборудовании. В противном случае будет использована изотропная фильтрация.

В строках 25–27 выполняется загрузка данных текстурного изображения из линейного массива байтов `bytes` в видеопамять. Читателю рекомендуется самостоятельно ознакомиться с назначением параметров функции загрузки по документации OpenGL. Следует отметить, что данная функция синхронна (не возвращает управление до тех пор, пока весь переданный массив не будет скопирован в видеопамять), в связи с чем сразу после этого вызова массив `bytes` может быть безопасно очищен или переиспользован для других нужд.

В строке 28 осуществляется автоматическая генерация MIP-пирамиды на основе заданного изображения, которое используется как первый (самый большой по площади) MIP-уровень.

Для того чтобы на этапе обработки фрагментов осуществить выборку цвета из текстуры, перед вызовом отрисовки необходимо сделать её активной и передать указатель на неё во фрагментный шейдер в качестве юниформ-переменной. Команды для этого приведены в листинге 2.

Листинг 2. Активация текстуры

```
1 glActiveTexture(GL_TEXTURE0 + texUnit);  
2 glBindTexture(GL_TEXTURE_2D, handler);  
3 glUniform1i(mapLocation, texUnit);
```

Переменная `mapLocation` имеет тип `GLint` и хранит идентификатор соответствующей юниформ-переменной. Из соображений оптимизации её следует означить на этапе загрузки графических ресурсов однократно:

```
mapLocation = glGetUniformLocation(program, "u_map");
```

Если на сцене используется всего одна текстура, её активация также может быть сделана однократно. Однако в общем случае, когда различные объекты сцены используют разные текстуры, действия по активации необходимо производить каждый раз заново.

Для организации доступа к текстурам из шейдера используются т. н. *текстурные блоки* (англ. *Texture Unit*). В целях оптимизации работы видеокарты во время отрисовки графический конвейер имеет доступ лишь к небольшому числу текстурных блоков. Это число значительно меньше максимального числа текстур, которые можно одновременно хранить в видеопамяти (так, например, на современных настольных компьютерах доступно, как правило, не более 32 текстурных блоков, но при этом в видеопамяти могут быть сохранены десятки тысяч различных текстур). Такой подход упрощает и ускоряет функции семплирования цвета.

Перед каждым вызовом отрисовки сначала активируется нужный текстурный блок (командой `glActiveTexture`, строка 1 листинга 2, где `texUnit` – номер текстурного блока). Затем осуществляется биндинг текстуры (командой `glBindTexture`, строка 2 листинга 2, где `handler` – идентификатор текстуры). В качестве значения юниформ-переменной в шейдер передаётся *порядковый номер текстурного блока* (а не идентификатор самой текстуры, строка 3 листинга 2).

Исходный код простейшего фрагментного шейдера, осуществляющего выборку цвета из текстуры, приведён в листинге 3. Для объявления юниформ-переменной, которая будет хранить ссылку на текстуру, использован специальный тип `sampler2D`, декларированный стандартом OpenGL как средство получения доступа к двумерным текстурным блокам с целью семплирования. Сама выборка осуществляется функцией `texture`, принимающей на вход ссылку на текстуру и вектор текстурных координат и возвращающей вектор цвета.

Листинг 3. Фрагментный шейдер для наложения текстуры

```
1 uniform sampler2D u_map;
2 in vec2 v_texCoord;
3 layout (position = 0) out vec4 o_fragColor;
4 void main()
```

```

5 {
6     o_fragColor = texture(u_map, v_texCoord);
7 }

```



Задание. Сгенерировать для ландшафта текстурные координаты с учётом того, что будет использован режим повторения. Загрузить из внешнего файла изображение, создать на его основе текстуру и наложить её на подготовленный ранее рельеф.

Подсказка. Для создания качественного ландшафта необходимо использовать бесшовную текстуру. Изображения для таких текстур без труда можно найти в сети Интернет.

1.3. Освещение ландшафта

Следующий важный шаг на пути к получению реалистично выглядящего ландшафта – моделирование освещения. *Модель освещения* (англ. *Shading Model*) – это конкретная схема определения интенсивности отражённого к наблюдателю света в каждой точке изображения. Основная идея состоит в том, чтобы путём изменения яркости цветов поверхности ландшафта имитировать на ней игру света и тени.

Различают локальные (англ. *Local*) и глобальные (англ. *Global*) модели освещения. Локальные модели учитывают только *прямое освещение* (англ. *Direct Lighting*) – свет, падающий на поверхность моделируемого объекта от источников. Глобальные же принимают во внимание и *непрямое освещение* (англ. *Indirect Lighting*) – свет, попадающий на поверхность моделируемого объекта вследствие отражения от других объектов сцены.

Как нетрудно догадаться, глобальные модели, хотя и более адекватно имитируют действительность, оказываются более сложными в реализации и затратными в плане использования вычислительных ресурсов. В силу этого они используются в основном в приложениях, работающих с графикой в пакетном режиме (напри-

мер, в фотореалистичных визуализаторах). В системах реального времени значительно чаще используются локальные модели.

Существует огромное множество различных локальных моделей освещения, пригодных для имитации разнообразных оптических свойств, присущих различным материалам, например, металлу, пластике, дереву, резине, ткани и т. п. Кроме того, специализированные локальные модели используются для осуществления различных визуальных эффектов с целью воссоздания требуемой стилистики сцены, например, мультипликационной рисовки, эффекта карандашного наброска и т. п.

1.3.1. Модель освещения Фонга

Одной из самых простых в реализации и в то же время достаточно гибких в плане настройки локальных моделей освещения является *модель Фонга* (англ. *Phong Shading*) [6]. У этой модели есть несколько модификаций, адаптирующих её для конкретных задач. Она предполагает, что оптические свойства освещаемой поверхности могут быть описаны как сочетание *матовости* и *глянца* в определённой (*неявно* задаваемой) пропорции. Несмотря на примитивность, эта модель хорошо подходит для имитации пластмассы, резины, гипса и глины, а при удачном подборе параметров способна имитировать оптические свойства и других материалов – камня, стекла, металла, дерева и т. п. (хотя чаще для этих материалов используются более сложные подходы).

В результате моделирования вычисляется итоговый цвет C , в который должна быть окрашена каждая точка P поверхности освещаемого объекта. Цвет определяется на основе суммы двух составляющих: *диффузного света* (англ. *Diffuse*) и *зеркального блика* (англ. *Specular*). Диффузный свет – это модель света, попавшего на поверхность из источника и равномерно рассеянного этой поверхностью во всех направлениях. Свойство равномерно рассеивать свет присуще матовым поверхностям. Зеркальный блик – это модель света, попавшего на поверхность из источника и отражённого в сторону наблюдателя. Свойство отражать свет в определённом направлении присуще глянцевым поверхностям.

В своём самом базовом виде модель Фонга предназначена для имитации освещения точечным всенаправленным источником света и имеет следующие параметры:

1. P – точка поверхности в пространстве.
2. L – точка, характеризующая положение источника света в пространстве.
3. E – точка, характеризующая положение наблюдателя в пространстве.
4. $\vec{n}$ – вектор внешней нормали к поверхности освещаемого объекта в точке P .
5. d_{min} – настроечный коэффициент, определяющий минимально допустимый уровень освещённости объекта в точке P .
6. s_{focus} – настроечный коэффициент, определяющий сфокусированность зеркального блика на поверхности освещаемого объекта в точке P .
7. c_{base} – базовый цвет поверхности освещаемого объекта в точке P .
8. c_{light} – цвет источника света.

Следует отметить, что у модели Фонга, как и у большинства других локальных моделей освещения, есть две основных модификации: моделирование на уровне вершин (иначе называемое *поверхшинным освещением*, англ. *Per-Vertex Shading*) и на уровне фрагментов (иначе называемое *пофрагментным освещением*, англ. *Per-Fragment Shading*). В первом случае цвет определяется для каждой вершины полигональной сетки освещаемого объекта, а затем билинейно интерполируется для фрагментов (точек растра) его полигонов. Во втором случае билинейной интерполяции подвергаются координаты вершин и нормалей, а цвет вычисляется для каждого фрагмента полигона индивидуально. Второй случай обеспечивает более высокое качество итогового изображения, однако при этом требует большего количества вычислений.

Геометрические построения, необходимые для вычисления цвета по модели Фонга, приведены на рис. 11.

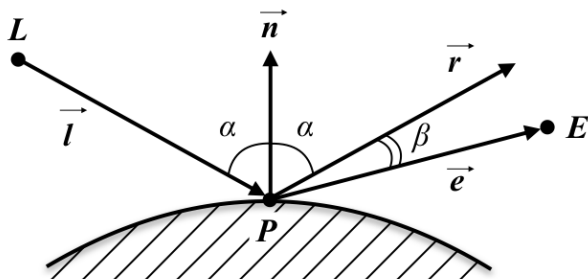


Рис. 11. Геометрические построения модели освещения Фонга

В основе вычисления интенсивности диффузного освещения по модели Фонга лежит *закон Ламберта* (англ. *Lambert's Law*): сила света, излучённого плоской рассеивающей площадкой в каком-либо направлении прямо пропорциональна остроте угла между этим направлением и перпендикуляром к площадке.

В интересах моделирования этот закон может быть переформулирован: освещённость точки поверхности прямо пропорциональна остроте угла между вектором, обратным вектору падения света, и нормалью к поверхности в данной точке. На рис. 11 упомянутый угол обозначен за α . В такой формулировке закон Ламберта легко согласуется с бытовым опытом: наилучшим образом точка поверхности освещена, если источник света находится «в зените», т. е. прямо над ней ($\alpha = 0$), и тем хуже, чем источник света «ближе к горизонту» ($|\alpha| \rightarrow \pi/2$). «Скрывшись за горизонтом» ($|\alpha| > \pi/2$), источник света перестаёт освещать данную точку поверхности напрямую.

Для численной оценки остроты угла используется его косинус. Чтобы получить коэффициент диффузного освещения точки, отрицательные значения косинуса отсекаются: отрицательная интенсивность освещения не имеет физического смысла. Порогом отсекаения в модели Фонга служит указанный ранее параметр d_{min} . Он, по сути, задаёт интенсивность т. н. *окружающей подсветки* (англ. *Ambient*), т. е. имитирует свет, отражённый от других объектов сцены. Однако наличие этого параметра не делает модель Фонга глобальной, так как его значение является наперёд заданной константой, а не вычисляет-

ся динамически на основе взаимного расположения источника света и объектов сцены. Тем не менее данный параметр позволяет избежать появления на поверхности моделируемого объекта абсолютно тёмных (чёрных) зон, тем самым повышая реалистичность результата визуализации.

Пусть $\vec{n}$ – вектор единичной длины (если это не так, его необходимо нормировать). Тогда, с учётом вышесказанного, коэффициент диффузного освещения $d \in [d_{min}, 1]$ вычисляется по формуле

$$d = \max \{ \cos \alpha, d_{min} \} = \max \{ -\vec{l} \cdot \vec{n}, d_{min} \}, \quad (1.1)$$

где $\vec{l} = \frac{P-L}{|P-L|}$ есть нормированный вектор падения света.

Для определения силы зеркального блика в точке P , по аналогии с законом Ламберта, оценивается острота угла между вектором $\vec{r}$ отражения света от поверхности в данной точке и осью зрения наблюдателя, выраженной вектором $\vec{e}$ (угол β на рис. 11).

Управление сфокусированностью блика осуществляется путём возведения косинуса угла β в степень s_{focus} . Чем эта степень больше, тем более чётко выражен блик, но и тем меньше его площадь.

Коэффициент зеркального блика s вычисляется следующим образом:

$$s = \max \{ \cos^{s_{focus}} \beta, 0 \} = \max \{ (\vec{r} \cdot \vec{e})^{s_{focus}}, 0 \}, \quad (1.2)$$

где $\vec{e} = \frac{E-P}{|E-P|}$,

$\vec{r} = \vec{l} - 2(\vec{n} \cdot \vec{l})\vec{n}$; в языке GLSL для получения вектора отражения существует специальная встроенная функция `reflect`.

Следует отметить, что получившийся коэффициент s_{focus} инвариантен относительно знака вектора $\vec{n}$, т. е. не изменится, если инвертировать нормаль. Это означает, что при использовании формулы (1.2) в текущем виде зеркальный блик будет появляться как на стороне поверхности, обращённой к источнику, так и на противоположной стороне. Для устранения этого нежелательного эффекта следует принудительно обнулять s , если $\cos \alpha < 0$.

Итоговый цвет C поверхности в точке P может быть определён по формуле

$$C = c_{light}(dc_{base} + s). \quad (1.3)$$

В результате таких вычислений значение каких-то компонентов цвета может оказаться больше 1, однако графический процессор автоматически ограничивает цветовые данные отрезком $[0, 1]$, поэтому никаких дополнительных действий производить не требуется; полученный цвет может быть присвоен соответствующему фрагменту.

Для корректности вычисления освещённости точки P , L и E должны быть выражены в одной системе координат, а вектор $\vec{n}$ – в базисе этой системы. Какая именно система координат будет использована – локальная система координат визуализируемого объекта, глобальная система координат сцены, система нормированных координат устройства, либо какая-то ещё – не важно, главное, чтобы она была *одной и той же* для всех обрабатываемых сущностей. При этом с точки зрения вычислений, как правило, проще всего использовать глобальную систему координат сцены. В этом случае координаты точек P , L могут быть вычислены следующим образом:

$$P = M_{object}v_{object}, \quad (1.4)$$

$$L = M_{light}v_{light}, \quad (1.5)$$

где M_{object} – матрица вида-модели объекта (отвечает за размещение объекта на сцене; *не содержит преобразований проекции*),

v_{object} – координаты обрабатываемой вершины объекта,

M_{light} – матрица вида-модели источника света (отвечает за размещение источника света на сцене; в самом простом случае – единичная),

v_{light} – координаты начального положения света (как правило, некоторая константа, определяемая программистом исходя из логики приложения).

Если для отображения сцены используется перспективная проекция, построенная классическим образом через угол обзора, ближнюю и дальнюю плоскости отсечения (см. [7]), то

$$E = \{0, 0, 0\}.$$

Несколько сложнее вычисляется вектор $\vec{n}$. Во-первых, должна быть определена нормаль к поверхности визуализируемого объекта. Во-вторых, эта нормаль должна быть трансформирована сообразно трансформации объекта (при размещении объекта на сцене). Решение этих задач рассмотрено в следующем подразделе.

1.3.2. Вычисление и трансформация нормалей

Часто координаты нормалей, наряду с координатами вершин, сохраняются в файл 3D-модели при её создании в соответствующем графическом редакторе трёхмерной графики. Такие редакторы, как правило, имеют в своём составе как функции автоматического вычисления нормалей к поверхностям моделируемых объектов, так и средства для модификации направлений нормалей вручную. В этом случае нормали считаются заданными наперёд. Однако в противном случае нормали должны быть вычислены.

Однозначного способа определения нормалей для произвольной 3D-модели, заданной лишь полигональной сеткой, не существует. Но обычно исходя из сути моделируемых объектов можно сделать некоторые предположения о их геометрической природе и соответственно с достаточной для обеспечения нужного визуального качества степенью точности определить нормали к их поверхностям.

Если для 3D-объекта известна порождающая функция $f(x, y, z)$ его поверхности (т. е. поверхность данного объекта есть множество точек $P(x, y, z)$, удовлетворяющих условию $f(P) = 0$), то нормаль в каждой точке P есть градиент (вектор, составленный из частных производных) функции f .

Однако в большинстве случаев порождающая функция неизвестна. Тогда нормали могут быть вычислены при помощи *алгоритма сглаживающих групп* (англ. *Smoothing Groups Algorithm*):

1. Для каждой вершины v полигональной сетки определяется «сглаживающая группа» S – множество полигонов (как правило, инцидентных этой вершине), составляющих участок поверхности, достаточный для определения кривизны в вершине v .

2. Для каждого полигона $p \in S$ определяется нормаль $\vec{n}_p$ к нему. Согласно требованиям, предъявляемым к обрабатываемым на видеокарте 3D-моделям, составляющие их полигоны должны быть планарными [7]. В связи с этим нахождение нормали к полигону сводится к нахождению нормали к соответствующей плоскости. Из двух возможных векторов нормали по некоторому правилу выбирается один, «наиболее подходящий» для решения задачи моделирования освещения.
3. Нормаль $\vec{n}_v$ для вершины v определяется как средневзвешенное нормалей $\vec{n}_p$:

$$\vec{n}_v = \sum_{p \in S} w_p \vec{n}_p, \quad (1.6)$$

где w_p – весовой коэффициент соответствующего полигона.

Определение сглаживающей группы для каждой вершины – в общем случае эвристический процесс, требующий дополнительных знаний о природе моделируемого объекта. Сглаживающая группа формирует на поверхности 3D-модели участок, выглядящий при освещении гладким (отсюда и название) за счёт соответствующей игры света и тени. Происходит это потому, что направление нормалей вдоль всей сглаживающей группы изменяется плавно, так как нормали интерполируются между вершинами соответствующих полигонов, а их направление в вершинах есть усреднение нормалей к полигонам. В связи с этим нужно знать, какие части поверхности нуждаются в сглаживании, а какие – нет. Например, если моделируется сфера, вся поверхность должна быть гладкой. Если же куб – переход светотени между гранями, скорее всего, должен быть резким. В случае моделирования цилиндра боковая поверхность должна быть сглаженной, но основания должны быть отделены от неё резким переходом.

В некоторых случаях информация о сглаживающих группах хранится среди данных, описывающих 3D-модель. Однако если такой информации нет, они могут быть определены по алгоритму *разделения рёбер* (англ. *Edge Split*):

1. Для каждой вершины v определяется множество I инцидентных ей полигонов.

2. Множество I разбивается на подмножества $I_1, I_2, \dots, I_n$ так, чтобы для каждой пары полигонов $p \in I_i, q \in I_i, i = \overline{1, n}$ выполнялось условие $\theta < \Theta$, где θ – двугранный угол между ними, Θ – параметр алгоритма.
3. Вершина v удаляется из 3D-модели. Вместо неё создаются n вершин v_i , каждая из которых занимает то же геометрическое место, что и v , но инцидентна лишь полигонам из множества I_i . При этом данное множество становится сглаживающей группой для v_i .

Данный алгоритм базируется на предположении о том, что поверхность 3D-модели достаточно точно аппроксимирована и каждый её участок можно классифицировать как гладкий либо заострённый по величине угла между смежными полигонами. Параметр Θ требует подбора в зависимости от конкретной задачи. Так, например, при $\Theta \leq \pi/2$ алгоритм успешно определит сглаживающие группы для цилиндра.

В самом простом (вырожденном) случае $\Theta = 2\pi$. Фактически это означает, что сглаживающей группой для вершины v назначается множество I , а шаги 2 и 3 могут быть полностью исключены.

Следует обратить внимание на то, что, поскольку нормали задаются в вершинах в виде атрибутов, для формирования резких границ между полигонами вершины 3D-модели необходимо дублировать. Так, например, куб, геометрически состоящий из 8 вершин, должен быть представлен моделью из 24 вершин, так как вершины каждой его грани имеют свои нормали, не совпадающие с вершинами смежных граней.

Следующая проблема – выбор «нужной» нормали к полигону. Для этого существует три основных варианта:

1. Выбор внешней (направленной «наружу» объекта) нормали.
2. Выбор внутренней (направленной «внутри» объекта) нормали.
3. Выбор нормали на основе порядка обхода вершин полигона.

Внешние нормали выбираются в том случае, если согласно логике приложения наблюдатель смотрит на данный объект снаружи. Такая ситуация является наиболее типичной.

Если объект незамкнут (как, например, ландшафт, построенный по карте высот), определение внешней нормали осуществляется на основе ориентации его поверхности относительно наблюдателя. Обычно с этой целью выбирается некоторое главное направление $\vec{n}'$, и из двух нормалей в точке P выбирается та, угол которой с $\vec{n}'$ меньше. Так, например, для ландшафта логично положить $\vec{n}' = \{0, 1, 0\}$.

Замкнутые объекты могут быть выпуклыми и невыпуклыми.

Если объект является выпуклым, определение внешней нормали сводится к проверке величины угла ϕ между вектором нормали $\vec{n}$ и вектором $\vec{n}' = P - C$, где P – анализируемая точка на поверхности объекта, C – точка геометрического центра объекта (среднее арифметическое координат всех его точек). Если $\cos \phi > 0$, нормаль является внешней, иначе – внутренней.

В случае невыпуклого объекта алгоритм усложняется. Например, может быть использован *алгоритм трассировки луча* (англ. *Ray Tracing*):

1. Из анализируемой точки P поверхности объекта в направлении проверяемой нормали $\vec{n}$ строится луч $P_{\vec{n}}$.
2. Определяются все пересечения луча $P_{\vec{n}}$ с полигонами объекта.
3. Если число найденных пересечений оказалось чётным, нормаль $\vec{n}$ – внешняя, иначе – внутренняя.

Внутренние нормали выбираются, если согласно логике приложения наблюдатель смотрит на данный объект изнутри (например, в роли объекта выступает модель дома, а наблюдатель должен находиться внутри него). Как правило, объектов такого рода на сцене значительно меньше чем тех, на которые следует смотреть снаружи. Внутренние нормали определяются аналогично внешним.

Несмотря на универсальность представленных выше методов, часто нормаль выбирается на основе порядка обхода вершин в полигоне: вектор нормали вычисляется как векторное произведение векторов, построенных по двум подряд идущим рёбрам полигона. Этот метод оказывается наиболее быстрым, хотя требует, чтобы порядок обхода вершин во всех полигонах 3D-модели соответствовал «правильной» (с точки зрения решаемой задачи) ориентации нормалей.

Такое требование, однако, является вполне обычным в задачах компьютерной графики и выполняется для большинства моделей, подготавливаемых в 3D-редакторах, генерируемых популярными алгоритмами и т. п. Дело в том, что на основе порядка обхода вершин может быть определена не только нормаль к полигону, но и установлено, какой стороной в данный момент полигон повёрнут к наблюдателю. Графические стандарты, в частности OpenGL, позволяют настроить различное поведение растеризации «передней» и «задней» сторон полигонов. На этом основана, например, техника *отсечения граней* (англ. *Face Culling*), позволяющая отключить растеризацию (и соответственно вычисление цветов фрагментов) полигонов, повёрнутых к наблюдателю задней или, наоборот, передней стороной.

Отсечение граней делается с целью уменьшения нагрузки на графический процессор без снижения качества результирующего изображения. Для замкнутых непрозрачных объектов (как выпуклых, так и невыпуклых) выполняется закон: при любом ракурсе наблюдатель, находясь снаружи объекта, будет видеть только наружные стороны его полигонов, а находясь внутри – только внутренние. В связи с этим, в соответствии с логикой приложения, часть граней объекта можно отбросить сразу после этапа преобразования вершин, когда известна проекция полигона и понятно, какой стороной он «лёг» на плоскость экрана, так как эти грани всё равно потом будут перекрыты другими. А поскольку вычисление цветов фрагментов, как правило, является ресурсоёмкой операцией, отказ от неё для заведомо невидимых частей поверхности даёт ощутимый прирост производительности. Для получения более подробной информации об отсечении задних граней можно прочитать документацию OpenGL по использованию константы `GL_CULL_FACE`.

В связи с этим 3D-модели обычно создаются таким образом, чтобы внешние стороны полигонов были ориентированы на наблюдателя (в зависимости от логики расположения модели относительно наблюдателя). Как правило, передней стороной полигона считается та, для которой обход вершин производится *против часовой стрелки* (англ. *Counterclockwise*, CCW). Следовательно, в большин-

стве случаев определение нормали в соответствии с порядком обхода вершин оказывается адекватным.

Как отмечалось выше, усреднение нормалей производится по формуле (1.6). Весовые коэффициенты w_p , как правило, выбирают таким образом, чтобы все нормали $\vec{n}_p$ вносили в $\vec{n}$ одинаковый вклад, а сама $\vec{n}$ оказалась в итоге единичной длины. Единичная длина нормали позволяет пользоваться формулами (1.1) и (1.2) без дополнительных нормировок (экономя тем самым такты графического процессора при обработке поверхностей). Исходя из этого формулу (1.6) можно записать так:

$$\begin{aligned}\vec{n}'_v &= \sum_{p \in S} \frac{\vec{n}_p}{|\vec{n}_p|}, \\ \vec{n}_v &= \frac{\vec{n}'_v}{|\vec{n}'_v|}.\end{aligned}\tag{1.7}$$

Нормали для ландшафта, построенного по карте высот, могут быть определены упрощённым алгоритмом сглаживающих групп (когда в качестве сглаживающей группы для каждой вершины берутся все смежные с ней полигоны). При условии, что полигональная сетка была создана так, чтобы порядок обхода вершин был всюду одинаков – против часовой стрелки, нормали могут быть определены исходя из этого порядка.

Сложность вычисления нормалей по такой упрощённой схеме оказывается $O(m)$, где m – число треугольников, составляющих поверхность 3D-модели. Итоговый алгоритм таков:

1. Первоначально в составе атрибутов каждой вершины v 3D-модели резервируются 3 вещественных значения для хранения трёхмерных координат нормали. Эти координаты инициализируются нулями.
2. Совершается обход индексов 3D-модели тройками (фактически – обход треугольников, составляющих её поверхность). Для каждой тройки индексов i_j, i_{j+1}, i_{j+2} , $j = \overline{1, n-2}$, где n – число индексов модели, выбираются соответствующие вершины $v_{i_j}, v_{i_{j+1}}, v_{i_{j+2}}$, составляющие очередной треуголь-

ник T_k , где $k = \frac{j+2}{3}$ – номер треугольника, соответствующего выбранным индексам.

3. Вычисляется нормаль $\vec{n}$ единичной длины (нормированная) к треугольнику T_k : $\vec{n}' = (v_{i_{j+1}} - v_{i_j}) \times (v_{i_{j+2}} - v_{i_{j+1}})$, $\vec{n} = \frac{\vec{n}'}{|\vec{n}'|}$.
4. К нормали, хранящейся в атрибутах каждой из вершин T_k , добавляется $\vec{n}$, результат нормируется и записывается в соответствующую вершину на место предыдущих координат нормали.

В результате выполнения этих шагов нормали как бы «аккумулируются» в вершинах, и к концу обхода будут представлять собой результат требуемого усреднения. За счёт постоянных нормировок длина нормалей поддерживается единичной, и каждый треугольник при вычислении вносит одинаковый вклад в соответствующие нормали, независимо от своей площади (без перенормировки треугольники большей площади отклоняли бы нормали сильнее, чем треугольники меньшей).

После того, как нормали вычислены, их необходимо трансформировать так, чтобы они остались перпендикулярны трансформированной поверхности 3D-модели.

Как отмечалось выше, преобразование вершин объекта для вычисления освещения обычно происходит по формуле (1.4). Матрица M_{object} изменяет систему координат объекта с локальной на глобальную, т. е. перемещает объект из его начального положения (и пространственной ориентации) на общую сцену. Эта матрица может заключать в себе произвольную смесь всех возможных аффинных преобразований.

На первый взгляд может показаться, что нормали нужно трансформировать при помощи этой же матрицы. Но это не так:

1. Аффинные преобразования включают параллельный перенос, масштабирование, поворот и наклон. Все эти операции определены для точек и изменяют их *положение*. Нормали же являются векторами, следовательно, для них определены только последние три операции, связанные с изменением *длины и направления*.

2. Все аффинные преобразования сохраняют параллельность прямых, однако масштабирование и наклон в общем случае нарушают перпендикулярность.

Следовательно, матрица M_{object} должна быть преобразована для получения из неё матрицы N трансформации нормалей. Правило преобразования схематично показано на рис. 12. Матрица трансформации нормалей (или просто *матрица нормалей*, англ. *Normal Matrix*) – инверсно-транспонированный первый главный минор третьего порядка матрицы вида-модели объекта. Математический вывод этого правила представлен в [8].

$$N = \left(\begin{pmatrix} m_0 & m_4 & m_8 \\ m_1 & m_5 & m_9 \\ m_2 & m_6 & m_{10} \end{pmatrix}^{-1} \right)^T$$

Рис. 12. Правило вычисления матрицы нормалей на основе матрицы вида-модели объекта

Обычно библиотеки работы с матрицами содержат в своём составе все необходимые функции для вычисления матрицы нормалей.



Задание. Вычислить для ландшафта нормали и реализовать освещение по модели Фонга.

Подсказка. С целью постепенного наращивания сложности сначала рекомендуется вычислить и трансформировать нормали и убедиться, что на этом этапе нет ошибок. Отлаживать вычисление нормалей можно, выводя модули их координат в виде цвета поверхности.

2. Моделирование неба

Следующей важной составляющей открытого пространства является *небо* (англ. *Sky*). В компьютерной графике под небом понимается пространственная структура, заполняющая фон сцены и *кажущаяся* бесконечно удалённой.

Существует несколько подходов к моделированию неба. Одним из самых популярных (ввиду своей простоты и достаточной универсальности) является т. н. *небесный куб* (англ. *Skybox*). Идея этого подхода заключается в формировании специальной текстуры, называемой *кубической картой* (англ. *Cubemap*) – совокупности из 6 изображений, представляющих собой развёртку куба, как показано на рис. 13.

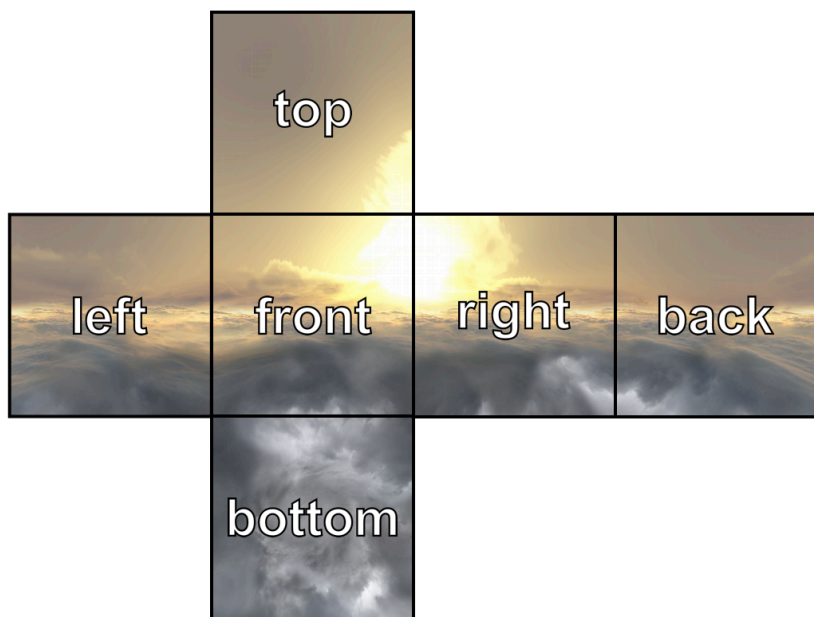


Рис. 13. Кубическая карта

Кубическая карта накладывается на параллелепипед, который визуализируется «вокруг» видимой сцены так, чтобы наблюдатель оказался в его центре, а стороны казались бесконечно удалёнными.

Для имитации эффекта бесконечной удалённости предпринимаются следующие меры:

1. Текстурированный параллелепипед выводится самым первым (до отправки на графический конвейер всех остальных объектов сцены).
2. При выводе параллелепипеда отключается z-буфер (полностью или только запись в него).
3. Модель параллелепипеда, как правило, делают под размер NDC, в случае использования OpenGL: $[-1; 1] \times [-1; 1] \times [-1; 1]$.
4. Вывод параллелепипеда осуществляется при помощи матрицы модели, содержащей только поворот (не содержащей ни перемещение, ни масштаб).

При формировании модели этого параллелепипеда следует учитывать, что для отображения на экран будет проецироваться его *внутренняя* поверхность, а не наружная (так как наблюдатель находится внутри): если включен режим отсечения задних граней, порядок обхода вершин в параллелепипеде должен быть выбран соответствующим образом.

Массив вершин и индексов для формирования модели небесного куба, пригодной для отображения средствами стандарта OpenGL, приведён в листинге 4.

Листинг 4. Массивы вершин и индексов для модели небесного куба

```
1 const GLfloat vertices[] =  
2 {  
3     -1.0f, -1.0f, 1.0f,  
4     1.0f, -1.0f, 1.0f,  
5     1.0f, 1.0f, 1.0f,  
6     -1.0f, 1.0f, 1.0f,  
7     -1.0f, -1.0f, -1.0f,  
8     1.0f, -1.0f, -1.0f,  
9     1.0f, 1.0f, -1.0f,
```

```

10      -1.0f, 1.0f, -1.0f
11  };
12  const GLuint indices[] =
13  {
14      0, 3, 1, 1, 3, 2,
15      0, 4, 7, 7, 3, 0,
16      1, 2, 6, 6, 5, 1,
17      5, 6, 7, 7, 4, 5,
18      3, 7, 6, 6, 2, 3,
19      0, 1, 5, 5, 4, 0
20  };

```

На современном оборудовании кубические карты имеют аппаратную поддержку и специальный механизм наложения, который по сравнению с традиционными «прямоугольными» текстурами оказывается более удобен и эффективен с точки зрения использования ресурсов видеокарты.

Кубическая карта составляется из отдельных изображений, последовательно загружаемых в видеопамять. В OpenGL для загрузки используется функция `glTexImage2D` (та же, что и для обычных двумерных текстур), однако первым параметром для каждой стороны куба указывается своя константа. Допустимые константы перечислены в табл. 1, в графе «Источник».

Наложение осуществляется по трёхкомпонентным текстурным координатам, которые представляют собой, по сути, координаты вектора, задающего направление относительно центра куба к его стороне для выборки цвета. Этот механизм легко представить себе в виде следующей метафоры: человек стоит в центре кубической комнаты, стены, пол и потолок которой оклеены фотообоями, и указывает в некотором направлении. Выборка цвета осуществляется из той области, на которую он указывает.

Каждый графический стандарт, описывающий кубические карты, определяет правило получения стороны куба и двумерных текстурных координат $\{s, t\}$ на ней для заданных координат на-

правления выборки $\{x, y, z\}$. В OpenGL для этого используются следующие формулы:

$$\begin{aligned} s &= \frac{1}{2} \left(\frac{s_c}{m} + 1 \right), \\ t &= \frac{1}{2} \left(\frac{t_c}{m} + 1 \right), \end{aligned} \quad (2.1)$$

где $m = \max \{|x|, |y|, |z|\}$,

s_c, t_c определяются по табл. 1 [3, раздел 3.9.10].

Таблица 1. Вычисление текстурных координат внутри кубической карты

Условие	Источник	s_c	t_c
$m = x , x > 0$	TEXTURE_CUBE_MAP_POSITIVE_X	$-z$	$-y$
$m = x , x < 0$	TEXTURE_CUBE_MAP_NEGATIVE_X	z	$-y$
$m = y , y > 0$	TEXTURE_CUBE_MAP_POSITIVE_Y	x	z
$m = y , y < 0$	TEXTURE_CUBE_MAP_NEGATIVE_Y	x	$-z$
$m = z , z > 0$	TEXTURE_CUBE_MAP_POSITIVE_Z	x	$-y$
$m = z , z < 0$	TEXTURE_CUBE_MAP_NEGATIVE_Z	$-x$	$-y$

Основным достоинством кубической карты является возможность использовать единственный текстурный слот для доступа сразу к 6 различным изображениям, каждое из которых может достигать 4096×4096 текселей. Способ же выборки по заданному направлению делает данный вид текстур удобным инструментом для имитации эффектов отражения и преломления. В этом случае в качестве кубической карты используется результат рендеринга окружения отражающего / преломляющего объекта, а текстурные координаты вычисляются на основании законов оптики.

Код настройки и загрузки кубической карты при помощи API OpenGL приведён в листинге 5.

Листинг 5. Настройка и загрузка кубической карты

```

1 glBindTexture(GL_TEXTURE_CUBE_MAP, handler);
2 glTexParameteri(GL_TEXTURE_CUBE_MAP,
3                 GL_TEXTURE_MAG_FILTER,
4                 GL_LINEAR);
```

```

5  glTexParameteri(GL_TEXTURE_CUBE_MAP,
6                      GL_TEXTURE_MIN_FILTER,
7                      GL_LINEAR);
8  glTexParameteri(GL_TEXTURE_CUBE_MAP,
9                      GL_TEXTURE_WRAP_S,
10                     GL_CLAMP_TO_EDGE);
11 glTexParameteri(GL_TEXTURE_CUBE_MAP,
12                     GL_TEXTURE_WRAP_T,
13                     GL_CLAMP_TO_EDGE);
14 #define loadCubic(side, bytes)           \
15     glTexImage2D(side, 0, internalFormat, \
16     width, height, 0,                    \
17     externalFormat, type, bytes);
18 loadCubic(GL_TEXTURE_CUBE_MAP_NEGATIVE_X, left);
19 loadCubic(GL_TEXTURE_CUBE_MAP_POSITIVE_X, right);
20 loadCubic(GL_TEXTURE_CUBE_MAP_NEGATIVE_Y, top);
21 loadCubic(GL_TEXTURE_CUBE_MAP_POSITIVE_Y, bottom);
22 loadCubic(GL_TEXTURE_CUBE_MAP_NEGATIVE_Z, back);
23 loadCubic(GL_TEXTURE_CUBE_MAP_POSITIVE_Z, front);

```

Для работы с кубическими картами в OpenGL имеется специальный тип текстуры, которому соответствует константа `GL_TEXTURE_CUBE_MAP`, она используется для активации текстурного объекта в строке 1.

Фильтрация настраивается в строках 2–7. Для кубических карт могут быть использованы все те же самые способы фильтрации, что и для обычных двумерных текстур.

Режим повторов настраивается в строках 8–13. Этот режим влияет на координаты, полученные *после* преобразования по формулам (2.1).

В строках 18–23 осуществляется загрузка 6 изображений. Для простоты предполагается, что все изображения имеют одинаковый размер и формат, поэтому загрузка оформлена в виде макроса. Названия массивов байтов, хранящих загружаемые изображения, соответствуют подписям сторон развёртки на рис. 13.

Поскольку размер проекции небесного куба на экране фиксирован, мипмэппинг для используемой кубической карты оказывается не нужен (хотя принципиально механизм мипмэппинга для кубических карт поддерживается).

Вершинный и фрагментный шейдеры, которые могут быть использованы для вывода и текстурирования небесного куба, приведены в листингах 6 и 7 соответственно. Для обращения к кубической текстуре используется специальный тип – `samplerCube`. Функция выборки цвета `texture` является перегруженной и в данном случае принимает на вход трёхмерные координаты направления выборки.

Листинг 6. Вершинный шейдер небесного куба

```
1 #version 330
2 uniform mat4 u_mvp;
3 in vec3 a_position;
4 out vec3 v_texCoord;
5 void main()
6 {
7     v_texCoord = vec3(-a_position.x,
8                       -a_position.y,
9                       a_position.z);
10    gl_Position = u_mvp * vec4(a_position, 1.0);
11 }
```

Листинг 7. Фрагментный шейдер небесного куба

```
1 #version 330
2 uniform samplerCube u_map;
3 in vec3 v_texCoord;
4 layout (location = 0) out vec4 o_fragColor;
5 void main()
6 {
7     o_fragColor = texture(u_map, v_texCoord);
8 }
```



Задание. Создать и отобразить небесный куб. Добавить небесный куб в программу, строящую ландшафт так, чтобы получилась целостная сцена открытого пространства.

Подсказка. Набор изображений для формирования кубической карты без труда можно найти в сети Интернет по запросу «Cubemap». Кроме того, в сети Интернет можно найти уроки, посвящённые созданию набора текстур для небесного куба при помощи 3D-редакторов и/или специализированного программного обеспечения.

3. Пространственная ориентация объектов

Пространственная ориентация объекта (англ. *Orientation* или *Angular Position*) – это расположение объекта в пространстве безотносительно к координатам его центра.

Понятие ориентации тесно связано с понятием *поворота*. С этой точки зрения поворот (англ. *Rotation*) – это трансформация, которая превращает одну ориентацию в другую. Процесс изменения ориентации во времени называется *вращением* (англ. *Spin*).

Существует несколько способов формального описания ориентации объектов. При этом имеют место два подхода: описание ориентации «напрямую» (т. е. описание состояния объекта) и через поворот относительно некоторой «опорной» ориентации, в качестве которой выступает либо изначальная, либо предшествующая (если имеет место цепочка последовательных поворотов). Начальную ориентацию объекта принято называть «нулевой», или «ориентацией по умолчанию».

В обоих подходах требуется наличие некоторой *системы отсчёта* (англ. *Reference System*), при помощи которой можно описывать положения точек целевого (располагаемого) объекта, а также различные вспомогательные сущности (например, направляющие векторы осей поворота и т. п.). Как правило, такой системой является либо локальная система координат рассматриваемого объекта, либо глобальная система координат трёхмерной сцены.

Не следует путать ориентацию и *направление*. Направление объекта (англ. *Direction*) может быть выражено одним вектором (чаще всего направление «вперёд»: например, направление взгляда человека), в то время как ориентация выражается двумя векторами (чаще всего направлениями «вперёд» и «вверх»: например, направление взгляда человека и направление, перпендикулярное макушке его головы, определяющее поворот головы вокруг оси взгляда).

Задание ориентации через два (обязательно линейно-независимых, т. е. неколлинеарных) вектора («вперёд» и «вверх», англ. *Direction-Up*) является наиболее популярным из «прямых» способов.

Этот способ удобен тем, что оперирует лишь пространственными координатами (посредством которых и задаются векторы). В случае, если задающие ориентацию векторы $\vec{d}$ и $\vec{u}$ перпендикулярны, выставление целевого объекта в соответствующую ориентацию может быть осуществлено совмещением базисных векторов его локальной системы координат с векторами $\vec{d}$, $\vec{u}$ и $\vec{d} \times \vec{u}$. Данный способ интуитивно понятен в контексте описания неизменной ориентации, однако осуществлять повороты с его помощью оказывается не так тривиально (для этого требуется вычислять отклонения векторов).

Для динамического изменения ориентаций значительно лучше подходит описание на основе поворотов. Такое описание, однако, оказывается неоднозначным: от одной ориентации к другой можно прийти различными поворотами (так, например, повороты на 90° по часовой стрелке и на 270° против часовой стрелки приводят к одинаковой результирующей ориентации).

В компьютерной графике чаще всего применяются следующие способы представления поворотов (и соответственно способы описания ориентации):

1. Ось и угол (англ. *Axis Angle*).
2. Матрица направляющих косинусов (матрица поворота, англ. *Rotation Matrix*).
3. Углы Эйлера (англ. *Euler Angles*).
4. Кватернионы (англ. *Quaternions*).

У каждого из способов есть свои плюсы и минусы. Поскольку все они служат для одной цели, у программиста есть возможность в динамике переходить от одного способа к другому, обеспечивая тем самым наиболее удобный и эффективный путь решения каждой конкретной задачи в процессе разработки графических приложений.

3.1. Ось и угол

Описание однократного поворота в трёхмерном пространстве хорошо ложится на концепцию задания оси и угла (следует отметить, что данный способ применим только для трёхмерного пространства: в двумерном пространстве и пространствах высших размерностей поворот задаётся не вокруг оси, а в плоскости). Ось, как правило, задаётся направляющим вектором единичной длины. Однако нахождение нужной оси для перехода от одной ориентации к другой не всегда тривиально, поэтому при комбинировании поворотов данный способ обычно не применяется. Кроме того, оперируя лишь осью и углом, не используя никаких других математических объектов и приёмов, крайне неудобно осуществлять преобразование целевого объекта.

3.2. Матрица направляющих косинусов

Классическим средством представления ориентаций, осуществления поворотов и организации вращения в компьютерной графике являются матрицы. Будучи универсальным аппаратом для преобразования целевых объектов, матричная алгебра имеет аппаратную поддержку на современных графических процессорах и активно используется на практике. Как правило, для осуществления над целевым объектом поворота, описанного тем или иным способом, сначала осуществляют переход к матричному представлению, а затем, используя аппаратную поддержку, преобразуют объект. Так, например, классический вид матрицы поворота определяется на основе оси и угла [7].

Как уже известно читателю, матрицы поворота допускают также удобное комбинирование поворотов, осуществляемое посредством их перемножения. Однако нетривиальным оказывается представление в матричном виде анимации вращения: попытка покомпонентной линейной интерполяции матриц, задающих две «граничных» ориентации, между которыми требуется совершить анимационный переход, приводит к одновременному изменению и ориентации, и масштаба объекта.

3.3. Углы Эйлера

Достаточно интуитивно-понятным способом задания ориентации являются углы Эйлера (в компьютерной графике, как правило, используется их частный случай – углы Тайта-Брайана) [7]. Этот способ хорошо согласуется с пространственными координатами объектов, так как вращение осуществляется вокруг координатных осей, и при фиксированных осях описание ориентации производится лишь заданием величин соответствующих углов. Благодаря этому данный способ хорошо сочетается с бытовым опытом человека, а записанная в терминах углов Эйлера ориентация имеет «человекочитаемый» вид: имея на руках лишь числа, без соответствующей визуализации, человеку тем не менее довольно легко представить, как будет выглядеть результат применения заданной ориентации к целевому объекту. В связи с этим углы Эйлера часто применяются, если числовое выражение ориентации необходимо вынести на пользовательский интерфейс (например, в 3D-редакторах или навигационных системах). Однако данный способ обладает и рядом серьёзных недостатков: небиективность задания ориентации, зависимость результата от порядка применения поворотов и возникновение шарнирного замка (англ. *Gimbal Lock*) [7].

Кроме того, при помощи углов Эйлера сложно осуществлять анимацию вращения. С одной стороны может показаться, что достаточно линейно проинтерполировать значения углов двух ориентаций, чтобы получить промежуточную. Однако на самом деле такой подход приводит к корректному результату лишь в частных случаях (например, когда ориентации отличаются друг от друга значением только одного угла), в общем же случае возникают аномалии: перемещение точек целевого объекта в процессе его вращения происходит не по кратчайшей дуге. В связи с этим для решения задач анимации углы Эйлера обычно не используются.

3.4. Кватернионы

Для решения основных проблем, связанных с углами Эйлера, используется аппарат *кватернионов*. Кватернион (от лат. *quaterni* – по четыре) – это гиперкомплексное число, представимое как формальная сумма вида

$$q = xi + yj + zk + w, \quad (3.1)$$

где $x, y, z, w \in \mathbb{R}$,

i, j, k – мнимые единицы, $i^2 = j^2 = k^2 = ijk = -1$.

Кватернионы, как система гиперкомплексных чисел, предложены У. Гамильтоном (*W. Hamilton*) в 1843 г. и представляют собой векторное четырёхмерное пространство над полем действительных чисел, обозначаемое символом $\mathbb{H}$. Эта система находит практическое применение в самых разных научных областях, в общем случае для описания изометрий трёх- и четырёхмерных евклидовых пространств. В компьютерной графике кватернионы используются для описания ориентаций, поворотов и вращений в трёхмерном пространстве (для этих же целей они применяются в механике и в робототехнике). В этом смысле кватернионы подобны матрицам направляющих косинусов, но, во-первых, они являются более экономичными (кватернион задаётся лишь 4 действительными числами, тогда как матрица направляющих косинусов для трёхмерного пространства состоит из 9 элементов), а во-вторых, сильно упрощают процесс анимации вращений.

Из-за своего «экзотического» происхождения (кватернионы – элементы мнимого гиперпространства, по аналогии с тем, как комплексные числа – элементы мнимой плоскости) кватернионы воспринимаются многими начинающими программистами как что-то очень сложное, требующее больших объёмов вычислений и глубоких познаний в области высшей математики. В действительности аппарат кватернионов прост и эффективен в использовании – безусловно, познания в области высшей математики никогда не бы-

вают лишними, но чтобы пользоваться кватернионами, достаточно знать лишь основы линейной алгебры и стереометрии.

С точки зрения структуры кватернион может быть представлен либо как 4 действительных числа $\{x, y, z, w\}$, либо как пара – трёхмерный вектор и скаляр $\{\vec{v}, w\}$. Многие математические операции с кватернионами удобнее выполнять, используя вторую форму представления, так как она допускает применение векторных операций (например, скалярное и векторное произведение), вследствие чего многие формулы приобретают более лаконичную запись.

При представлении поворота кватернион фактически выражает ось и угол, что делает его близким соответствующему альтернативному способу выражения ориентаций. Различие состоит в том, что компоненты оси и угла входят одновременно во все 4 компонента кватерниона:

$$q = \left\{ \vec{a} \sin \frac{\phi}{2}, \cos \frac{\phi}{2} \right\}, \quad (3.2)$$

где $\vec{a}$ – вектор, задающий ось поворота,
 ϕ – угол поворота.

Такое представление обеспечивает удобное применение и изменение описываемых кватернионами поворотов посредством специфических кватернионных операций. Рассмотрим основные из этих операций и их смысл в контексте представления ориентаций.

Норма (англ. *Norm*) кватерниона вычисляется по формуле

$$N(q) = x^2 + y^2 + z^2 + w^2. \quad (3.3)$$

Длина (англ. *Magnitude*) кватерниона аналогична длине вектора:

$$|q| = \sqrt{N(q)}. \quad (3.4)$$

Кватернион, длина которого равна 1, называется *единичным* (англ. *Unit*) или *нормированным* (англ. *Normalized*), а иногда используют специальный термин – *версор* (англ. *Versor* от лат. «вращатель»). Использование в расчётах только единичных кватернионов позволяет реализовать некоторые операции более эффективно.

Сопряжённый (англ. *Conjugate*) кватернион определяется по формуле

$$q' = \{-\vec{v}, w\}. \quad (3.5)$$

Обратный (англ. *Inverse*) кватернион может быть получен по формуле

$$q^{-1} = \frac{q'}{N(q)}. \quad (3.6)$$

Обратный кватернион, по аналогии с матрицей направляющих косинусов, выражает обратный поворот. Как можно видеть, для единичного кватерниона обратный является сопряжённым и может быть получен простой сменой знака первых трёх компонентов.

Умножение Грассмана (часто называемое просто умножением, англ. *Multiplication*) осуществляется по формуле

$$q_1 q_2 = \{\vec{v}_1 \times \vec{v}_2 + w_1 \vec{v}_2 + w_2 \vec{v}_1, w_1 w_2 - \vec{v}_1 \cdot \vec{v}_2\}. \quad (3.7)$$

Результатом умножения Грассмана является кватернион, описывающий комбинацию поворотов, заданных кватернионами-операндами (по аналогии с перемножением матриц направляющих косинусов). Умножение Грассмана *некоммутативно* (как и сама выражаемая с его помощью последовательность поворотов).

Скалярное произведение (англ. *Dot Product*) по аналогии с обычными векторами вводится так:

$$q_1 \cdot q_2 = x_1 x_2 + y_1 y_2 + z_1 z_2 + w_1 w_2. \quad (3.8)$$

Для скалярного произведения имеет место следующее соотношение:

$$q_1 \cdot q_2 = |q_1| |q_2| \cos \frac{\phi}{2}, \quad (3.9)$$

где ϕ – угол поворота между ориентациями, которые задают q_1 и q_2 ; угол ϕ кратчайший, если $q_1 \cdot q_2 > 0$.

Умножение кватерниона на скаляр осуществляется покомпонентно, как умножение на скаляр обычного вектора:

$$sq = \{sx, sy, sz, sw\}, \quad s \in \mathbb{R}. \quad (3.10)$$

Сложение кватернионов также осуществляется покомпонентно:

$$q_1 + q_2 = \{x_1 + x_2, y_1 + y_2, z_1 + z_2, w_1 + w_2\}. \quad (3.11)$$

Зафиксируем на целевом объекте, ориентация которого задаётся кватернионами q_1, q_2 и $q = q_1 + q_2$, некоторую точку P . За P_q, P_{q_1} и P_{q_2} обозначим точки, в которые переходит точка P при соответствующих ориентациях объекта. Тогда справедливо утверждение, что точки P_q, P_{q_1} и P_{q_2} лежат в одной плоскости. Иными словами, все повороты, обеспечивающие переход между ориентациями q_1, q_2 и q имеют общую ось (и соответственно осуществляются в одной плоскости).

Данное свойство очень важно и является едва ли не самой привлекательной особенностью кватернионов. Из него вытекает утверждение, что линейная комбинация вида

$$q = s_1 q_1 + s_2 q_2, \quad s_1, s_2 \in \mathbb{R}, \quad q_1, q_2 \in \mathbb{H} \quad (3.12)$$

задаёт ориентацию, поворот от начальной к которой лежит в одной плоскости с поворотом от ориентации q_1 к q_2 . Если за s_1 и s_2 взять $(1 - t)$ и $t, t \in [0; 1]$ соответственно, а также $q_1 \cdot q_2 > 0$, все получившиеся ориентации, выраженные кватернионами q_t , будут принадлежать *кратчайшему* повороту от q_1 к q_2 . Иными словами (пользуясь предыдущими обозначениями), все точки P_{q_t} целевого объекта будут лежать на кратчайшей дуге, соединяющей точки P_{q_1} и P_{q_2} .

Вышесказанное можно переформулировать более лаконично: *линейная интерполяция* (англ. *Lerp*) кватернионов, скалярное произведение которых больше нуля, позволяет осуществить кратчайший поворот между соответствующими ориентациями. Если скалярное произведение меньше нуля, один из кватернионов (любой) следует умножить на -1 .

Это правило активно используется при анимации вращений, однако у него есть одна важная оговорка: при линейном изменении параметра t скорость вращения *нелинейна*.

Для линеаризации скорости вращения применяется *сферическая линейная интерполяция* (англ. *Slerp*):

$$q_t = \frac{\sin((1-t)\Omega)}{\sin \Omega} q_1 + \frac{\sin(t\Omega)}{\sin \Omega} q_2, \quad (3.13)$$

где $t \in [0; 1]$ и изменяется линейно,

Ω – половина угла кратчайшего поворота между ориентациями, которые задают q_1 и q_2 ; $\cos \Omega = \frac{q_1 \cdot q_2}{|q_1||q_2|}$, если $q_1 \cdot q_2 > 0$.

Как можно заметить, сферическая линейная интерполяция значительно менее эффективна с точки зрения производительности, так как требует вычисления тригонометрических функций. В связи с этим на практике вместо неё часто используется обычная линейная интерполяция, так как максимальная погрешность между данными способами вычисления промежуточных значений поворота составляет всего около 4° (максимальная погрешность имеет место при $\Omega = 90^\circ$ и уменьшается с уменьшением Ω). Сферическую линейную интерполяцию рекомендуется использовать только в тех случаях, когда нужна высокая точность промежуточных значений поворота. Для решения же рядовых задач анимации обычной линейной интерполяции зачастую бывает вполне достаточно.

Кватернион q может быть напрямую использован для поворота целевого объекта. С этой целью каждая точка объекта интерпретируется как радиус-вектор $\vec{p}$, и этот радиус-вектор поворачивается при помощи следующего алгоритма:

1. Радиус-вектор $\vec{p}$ преобразуется в кватернион путём добавления нуля в качестве четвёртого компонента: $q_p = \{\vec{p}, 0\}$.
2. Вычисляется произведение $q_t = q q_p q^{-1}$.
3. Полученный кватернион q_t преобразуется в радиус-вектор $\vec{p}_t$ путём отбрасывания четвёртого компонента. Полученный радиус-вектор соответствует радиус-вектору $\vec{p}$, повернутому кватернионом q .

Нетрудно показать, что кватернион вида

$$q_e = \{0, 0, 0, 1\} \quad (3.14)$$

не изменяет целевой объект, т. е. выражает «нулевой» поворот. Такой кватернион называют *тождественным* (англ. *Identical*). Обычно тождественный кватернион является значением по умолчанию при инициализации соответствующих структур данных в библиотеках, предоставляющих функции работы с кватернионами.

При использовании в графических приложениях кватернионы часто нужно конвертировать в матрицы направляющих косинусов. Так, например, графические процессоры не имеют аппаратного ускорения для работы с кватернионами (тогда как реализуют основные операции для матриц). Код преобразования кватерниона в матрицу приведён в листинге 8.

Листинг 8. Преобразование кватерниона в матрицу

```
1 mat4 matrixWithQuaternion(const quat &q)
2 {
3     float s = 2.0f / q.norm();
4
5     float x = q.x * s;
6     float y = q.y * s;
7     float z = q.z * s;
8     float xx = q.x * x;
9     float xy = q.x * y;
10    float xz = q.x * z;
11    float yy = q.y * y;
12    float yz = q.y * z;
13    float zz = q.z * z;
14    float wx = q.w * x;
15    float wy = q.w * y;
16    float wz = q.w * z;
17
18    mat4 result;
19
20    result[0] = 1.0f - (yy + zz);
21    result[1] = xy - wz;
22    result[2] = xz + wy;
23    result[3] = 0.0f;
```

```

24
25     result[4] = xy + wz;
26     result[5] = 1.0f - (xx + zz);
27     result[6] = yz - wx;
28     result[7] = 0.0f;
29
30     result[8] = xz - wy;
31     result[9] = yz + wx;
32     result[10] = 1.0f - (xx + yy);
33     result[11] = 0.0f;
34
35     result[12] = 0.0f;
36     result[13] = 0.0f;
37     result[14] = 0.0f;
38     result[15] = 1.0f;
39
40     return result;
41 }

```

В приведённой функции тип `mat4` обозначает матрицу 4×4 , которая, однако, используется в данном контексте только как матрица поворота (значащим является только первый главный минор третьего порядка). Для матрицы определена операция доступа к элементам исходя из того, что в памяти она хранится в виде линейного массива элементов типа `float`, причём уложенного по столбцам. Таким образом, например, элемент с номером 1 соответствует второму элементу первого столбца. Такой способ хранения матриц соответствует стандарту OpenGL.

Тип `quat` выражает кватернион, для которого определены операция `norm` подсчёта нормы (по формуле (3.3)) и поля `x`, `y`, `z`, `w`, хранящие соответствующие компоненты. Компоненты также имеют тип `float` (одинарная точность связана с тем, что современные графические процессоры, как правило, используют именно 32-битное представление вещественных чисел).

Как можно заметить, от деления в строке 3 можно избавиться, если известно, что `q` — единичный кватернион.

Преобразование кватерниона в матрицу очень эффективно с точки зрения производительности, так как использует исключительно арифметические операции (даже в том случае, если кватернион не единичный и нужно вычислить его норму).

Избежать шарнирного замка можно путём «аккумулирования» поворота в кватернионе: все повороты осуществляются путём домножения кватерниона, выражающего *текущую* ориентацию, на кватернион, обеспечивающий переход к новой. В этом случае, однако, рекомендуется дополнительно осуществлять принудительную перенормировку результата (приведение его к единичному кватерниону путём деления его на длину), чтобы избавиться от аккумуляции погрешности чисел с плавающей точкой.



Задание 1. Самостоятельно изучить вопрос преобразования матрицы направляющих косинусов в кватернион, углов Эйлера в кватернион и нахождения кватерниона по двум заданным векторам как кратчайшего поворота от одного вектора к другому.

Задание 2. Создать микробιβлиотеку для работы с кватернионами, в которой были бы реализованы все описанные в данном разделе операции.

Подсказка к заданию 2. Кватернионы в компьютерной графике, по аналогии с векторами и матрицами, эффективнее всего хранить в виде линейных (четырёхкомпонентных) массивов чисел типа `float`. В листинге 8 компоненты представлены отдельными публичными полями, но это сделано лишь для единообразия записи в пределах данного пособия.

Задание 3. Используя созданную бιβлиотеку, сравнить результаты анимации вращения, осуществляемого посредством сферической линейной и обычной линейной интерполяций кватернионов.

4. Моделирование камеры

При работе в трёхмерном пространстве человеку (как программисту, так и пользователю) удобно использовать метафору камеры – объекта, характеризующего положение наблюдателя в пространстве, а также управляющего свойствами «поля зрения» наблюдателя. С точки зрения внутренней организации графического конвейера ни камеры, ни наблюдателя не существует: есть лишь «опорная» система координат NDC, связанная с портом просмотра (который в свою очередь является, по сути, лишь правилом отображения содержимого буфера кадра на поверхности рендеринга [7]). Однако на более высоком уровне – на уровне графического движка или даже на уровне логики приложения – камера может быть представлена в виде отдельной сущности, упрощающей пространственную навигацию.

Если речь идёт об играх, симуляторах или 3D-редакторах, камера выступает необходимым элементом человеко-машинного интерфейса, обеспечивающим интуитивное управление ракурсом просмотра трёхмерных сцен.

В общем случае камера определяет не только позицию и пространственную ориентацию наблюдателя, но и характеристики моделируемой оптической системы: угол обзора, искажения, вносимые «линзой», глубину резкости, особенности цветопередачи и т. п.

Однако для упрощения рассмотрим лишь самые основные характеристики: положение, угол обзора и дальность видимости. В этом случае камера может быть представлена следующей матрицей:

$$C = PV, \quad (4.1)$$

где P – матрица проекции,

V – матрица вида.

Матрица проекции отвечает за параметры видимой области пространства: угол обзора, ближнюю и дальнюю плоскости отсечения (формулы построения этой матрицы приведены в [7]). Матрица

вида определяет ракурс сцены. Эту матрицу саму по себе часто называют матрицей камеры, хотя, строго говоря, более правильно называть так результирующую матрицу C . Преобразование объектов сцены при помощи этой матрицы происходит посредством преобразования каждой вершины каждого объекта по формуле

$$v' = CMv, \quad (4.2)$$

где M – матрица трансформации объекта,

v – исходные координаты вершины объекта,

v' – трансформированные координаты вершины объекта.

С точки зрения организации и решаемых навигационных задач различают следующие основные типы камер:

1. *Камера от первого лица* (англ. *First Person Camera*) – камера, имитирующая вид «из глаз» виртуального персонажа.
 2. *Камера от третьего лица* (англ. *Third Person Camera*) – камера, имитирующая слежение за виртуальным персонажем «со стороны».
 3. *Орбитальная камера* (англ. *Arcball Camera*) – камера, имитирующая облёт неподвижного объекта «по орбите» или свободное вращение объекта перед неподвижным наблюдателем.
- Рассмотрим реализацию каждого из типов более подробно.

4.1. Камера от первого лица

Камера от первого лица является самым важным видом камеры в контексте создания виртуальных миров и приложений виртуальной реальности, так как по своей природе максимизирует эффект присутствия пользователя на виртуальной сцене, позволяя ему видеть синтезированный мир «глазами» виртуального персонажа. Соответственно этот вид также хорошо подходит для работы со специализированными средствами обеспечения погружения, такими как шлемы виртуальной реальности, позволяя пользователю управлять ракурсом при помощи поворотов головы.

Матрица вида для камеры от первого лица строится по правилу «противодействия»: поскольку эта матрица технически описывает преобразование сцены, а не наблюдателя, она должна содержать преобразования, обратные преобразованиям наблюдателя. Так, например, если наблюдатель сдвигается вперёд, матрица вида должна включить в себя смещение сцены назад; если наблюдатель поворачивается направо, в матрицу вида добавляется поворот налево.

Организация камеры от первого лица, как правило, включает в себя поддержку некоторого набора команд по изменению положения наблюдателя, которые может отдавать пользователь, чтобы осуществлять навигацию по трёхмерной сцене и исследовать её «глазами» виртуального персонажа. В зависимости от решаемых задач набор команд может различаться, однако наиболее часто реализуются следующие:

1. *Шаг прямо* (англ. *Move*) – смещение виртуального персонажа вперёд/назад в направлении взгляда (с учётом ограничений субстрата, например, по поверхности ландшафта).
2. *Шаг вбок* (англ. *Strafe*) – смещение виртуального персонажа влево/вправо с сохранением направления взгляда (с учётом ограничений субстрата).
3. *Рыскание* (англ. *Yaw*) – поворот виртуального персонажа налево/направо.
4. *Тангаж* (англ. *Pitch*) – поворот виртуального персонажа вверх/вниз.
5. *Крен* (англ. *Roll*) – поворот виртуального персонажа набок по/против часовой стрелки.

Наиболее сложной частью реализации камеры является организация поворотов: команды 3–5. Для практической реализации этих команд могут быть использованы различные математические модели, наиболее популярными из которых являются углы Тайта-Брайана и кватернионы. Команды 1 и 2 в свою очередь зависят от команд 3–5 в силу привязки к пространственной ориентации.

Рассмотрим реализацию камеры на основе углов Тайта-Брайана.

Чтобы избежать инверсии направления поворота и шарнирного замка на значения углов накладывают некоторые ограничения, имитируя там самую ограниченную подвижность виртуального персонажа (так, например, человек в норме не может ходить вниз головой – в связи с этим тангаж и крен часто ограничивают некоторым довольно узким диапазоном допустимых значений).

При использовании углов Тайта-Брайана камера как объект содержит позицию (обозначим её `m_position`), выраженную тремя пространственными координатами `x`, `y`, `z`, и ориентацию (обозначим её `m_orientation`), выраженную тремя углами `yaw`, `pitch`, `roll`. Все компоненты имеют тип `float`. В листинге 9 приводится реализация вышеуказанных команд для данного случая.

Листинг 9. Реализация основных команд камеры от первого лица на основе углов Тайта-Брайана

```
1 void Camera::move(float delta)
2 {
3     m_position.x -= sin(m_orientation.yaw) * delta;
4     m_position.z -= cos(m_orientation.yaw) * delta;
5     m_terrain->constrain(m_position, m_height);
6 }
7 void Camera::strafe(float delta)
8 {
9     float a = m_orientation.yaw - PI / 2.0f;
10    m_position.x -= sin(a) * delta;
11    m_position.z -= cos(a) * delta;
12    m_terrain->constrain(m_position, m_height);
13 }
14 void Camera::yaw(float angle)
15 {
16     m_orientation.yaw = wrap(m_orientation.yaw -
17                             angle,
18                             0.0f, 2.0f * PI);
19 }
```

Функция `move` принимает на вход величину шага `delta`, которая должна корректироваться исходя из производительности системы визуализации так, чтобы всегда обеспечивать константную скорость перемещения (этот вопрос освещён в [7]). Значение величины шага может быть как положительным, так и отрицательным, и тем самым будут осуществляться смещения наблюдателя вперёд и назад соответственно. Скорректированные в строках 3–4 значения координат наблюдателя затем дополнительно изменяются в методе `constrain`, предоставляемом ландшафтом (ссылка на который хранится в переменной `m_terrain`). Возможная реализация этого метода приведена в листинге 10. Его цель – обеспечить удовлетворение ограничений, накладываемых субстратом на передвижение наблюдателя. Следует отметить, что данная реализация ориентирована на создание виртуального персонажа, «привязанного» к ландшафту, т. е. перемещающегося по некоторой поверхности. Именно поэтому движение вперёд и назад учитывает только угол рыскания.

Листинг 10. Реализация функции ограничения перемещения по субстрату

```
1 void Terrain::constrain(vec3 &position, float h)
2     const
3 {
4     if (position.x < 0.0f)
5         position.x = 0.0f;
6     else if (position.x >
7             (float)(m_size.height - 1))
8         position.x = (float)(m_size.height - 1);
9
10    if (position.z < 0.0f)
11        position.z = 0.0f;
12    else if (position.z >
13            (float)(m_size.width - 1))
14        position.z = (float)(m_size.width - 1);
15
16    position.y = h + height(position.x,
17                            position.z);
18 }
```

```

19 float Terrain::height(float x, float z) const
20 {
21     int i = (int)z;
22     int j = (int)x;
23     float fi = z - (float)i;
24     float fj = x - (float)j;
25     float nfi = 1.0f - fi;
26     float nfj = 1.0f - fj;
27     return ((heightAt(i, j) * nfi +
28             heightAt(i + 1, j) * fi) * nfj +
29             (heightAt(i, j + 1) * nfi +
30             heightAt(i + 1, j + 1) * fi) * fj);
31 }

```

Предполагается, что ландшафт организован в соответствии с описанием из главы 1 и имеет размер, хранящийся в переменной `m_size` и выраженный числом узлов сетки по длине и ширине.

Функция `constrain` принимает на вход позицию наблюдателя `position` на поверхности ландшафта и «рост» наблюдателя `h` (т. е. высоту положения его «глаз» над ландшафтом), а затем изменяет позицию в соответствии с ограничениями, не давая наблюдателю выйти за пределы ландшафта (строки 4–14) и корректируя фактическую высоту на основании высоты рельефа (строки 16–17).

Корректировка высоты наблюдателя основана на вычислении высоты рельефа в заданной точке поверхности ландшафта, что осуществляется функцией `height`. Данная функция производит билинейную интерполяцию значений высоты четырёх узлов сетки ландшафта, ближайших к заданной точке. Запрос высоты по целочисленным координатам узла осуществляется функцией `heightAt`. Её реализация не приводится, так как, по сути, она представляет собой тривиальное обращение к массиву значений, полученных из карты высот.

Функция `strafe` из листинга 9 аналогична по своей структуре функции `move`. Единственное отличие заключается в направлении движения: оно перпендикулярно текущему направлению «вперёд».

Функции для изменения углов рысканья, тангажа и крена полностью аналогичны, поэтому в листинге приведена только одна из них – функция `yaw`. Она принимает на вход величину изменения угла `angle`, которая, как и величина шага, может быть положительной и отрицательной для поворотов по и против часовой стрелки (связь знака с направлением поворота зависит от принятого в разрабатываемой системе соглашения), а также должна учитывать производительность для поддержания одинаковой скорости вращения. Угол корректируется в соответствии с правилом «противодействия», путём вычитания величины изменения, а не сложения с ней. При этом величина угла удерживается в диапазоне $[0; 2\pi]$, чтобы избежать переполнения. Для этого используется функция `wrap`, реализация которой приведена в листинге 11.

Листинг 11. Реализация функции «зацикливания» значений

```
1 float wrap(float a, float min, float max)
2 {
3     a -= min;
4     max -= min;
5     if (max == 0)
6         return min;
7     a = (float)fmod(a, max) + min;
8     if (a < min)
9         a += max;
10    return a;
11 };
```

В своей реализации функция `wrap` использует стандартную функцию `fmod` (доступную в библиотеке `cmath`), вычисляющую дробный остаток от деления заданных аргументов друг на друга.

Функция получения матрицы вида для камеры от первого лица, работающей на основе углов Тайта-Брайана, приведена в листинге 12. На основе матрицы вида по формуле (4.1) затем может быть вычислена матрица камеры, а с её помощью, по формуле (4.2) – образованы объекты сцены.

Листинг 12. Получение матрицы вида для для камеры от первого лица
на основе углов Тайта-Брайана

```
1 mat4 Camera::viewMatrix() const
2 {
3     return mat4::rotationZ(m_orientation.roll) *
4         mat4::rotationX(m_orientation.pitch) *
5         mat4::rotationY(m_orientation.yaw) *
6         mat4::translation(-m_position.x,
7                             -m_position.y,
8                             -m_position.z);
9 }
```

Фактически в функции `viewMatrix` осуществляется перемножение 4 матриц: матриц поворота вокруг координатных осей на соответствующие углы (строки 3–5) и матрицы переноса на величины, определяемые через позицию камеры, взятую с обратным знаком (строки 6–8). Знак «минус» при координатах позиции и обратный порядок сомножителей обусловлен описанным ранее правилом «противодействия».

Теперь рассмотрим организацию камеры от первого лица на основе кватернионов. Хотя с точки зрения вычислительной сложности этот вариант немного накладнее предыдущего, ему часто отдают предпочтение, так как:

1. При условии аккумуляции поворотов в кватернионе отсутствует проблема шарнирного замка.
2. Решение оказывается достаточно гибким, в частности, чтобы перейти от вращения вокруг осей локальной к вращению вокруг осей глобальной системы координат, достаточно лишь поменять порядок операндов при умножении кватернионов.
3. Значительно упрощается анимация поворотов.

В листинге 13 приводится реализация основных команд камеры от первого лица, работающей на основе кватернионов. В отличие от предыдущего случая переменная `m_orientation` является кватернионом, т. е. содержит 4 вещественных компонента x , y , z , w , а также располагает соответствующими операциями.

Листинг 13. Реализация основных команд камеры от первого лица
на основе кватернионов

```
1  void Camera::move(float delta)
2  {
3      vec3 dir(0.0f, 0.0f, 1.0f);
4      quat q = m_orientation;
5      q.w = -q.w;
6      q.rotateVector(dir);
7      m_position.x -= dir.x * delta;
8      m_position.z -= dir.z * delta;
9      m_terrain->constrain(m_position);
10 }
11 void Camera::strafe(float delta)
12 {
13     vec3 dir(-1.0f, 0.0f, 0.0f);
14     quat q = m_orientation;
15     q.w = -q.w;
16     q.rotateVector(dir);
17     m_position.x -= dir.x * delta;
18     m_position.z -= dir.z * delta;
19     m_terrain->constrain(m_position);
20 }
21 void Camera::yaw(float angle)
22 {
23     vec3 a(0.0f, 1.0f, 0.0f);
24     m_orientation = quat::fromAxisAngle(a, angle) *
25                     m_orientation;
26     m_orientation.normalize();
27 }
```

Команды `move` и `strafe`, как и в предыдущем случае, организованы аналогично друг другу, однако, как можно заметить, при использовании кватернионов вычисление направления движения (вектор `dir`) принимает значительно более интуитивно-понятный вид: задаётся направление движения в локальной системе координат наблюдателя (строки 3 и 13), затем преобразуется кватернионом, обрат-

ным выражающему ориентацию наблюдателя (строки 4–6 и 14–16), в результате чего получается направление движения в глобальной системе координат, при помощи которого можно корректировать позицию наблюдателя (строки 7–8 и 17–18). Инверсия кватерниона производится путём изменения знака его четвёртого (скалярного) компонента. Здесь предполагается, что кватернион единичный (версор). Функция `rotateVector` реализована в соответствии с алгоритмом, приведённом в разделе 3.4.

Команды поворота `yaw`, `pitch` и `roll` организованы аналогично друг другу и отличаются лишь осью, вокруг которой производится вращение. В листинге приведена лишь команда `yaw`. Ось поворота `a` задаётся в явном виде при помощи направляющего единичного вектора (строка 23). Затем на основе этой оси и заданного угла `alpha` строится кватернион, на него умножается текущий, и результат становится текущим (строки 24–25). Функция `fromAxisAngle` реализована на основе формулы (3.2). Чтобы избежать аккумуляции погрешности вещественных чисел, результат нормируется (строка 26). Нормировка осуществляется путём деления каждого компонента на длину, вычисляемую по формуле (3.4).

Вращения камеры вокруг глобальных осей можно добиться, поменяв местами сомножители в строках 24–25.

Функция получения матрицы вида приведена в листинге 14.

Листинг 14. Получение матрицы вида для для камеры от первого лица
на основе кватернионов

```
1 mat4 Camera::viewMatrix() const
2 {
3     return matrixWithQuaternion(m_orientation) *
4         mat4::translation(-m_position.x,
5                           -m_position.y,
6                           -m_position.z);
7 }
```

Реализация функции `matrixWithQuaternion` приведена в листинге 8.

Организация камеры от первого лица на основе углов Тайта-Брайана удобна в том случае, если моделируется антропоморфный персонаж и рассматривается поворот на уровне всего его корпуса. Кватернионы же позволяют достаточно адекватно смоделировать повороты головы. В сложных системах виртуальной реальности оба метода могут быть скомбинированы для обеспечения более полного и всестороннего контроля над ракурсом.



Задание. Реализовать оба вида камеры от первого лица и сравнить полученные результаты. Выбрать наиболее адекватный вид для навигации по виртуальному миру.

Подсказка. Команды движения вперёд/назад обычно связывают с клавишами на клавиатуре, тогда как повороты можно сделать как клавишами, так и движением курсора мыши.

4.2. Камера от третьего лица

Камера от третьего лица используется в том случае, если нужно либо зафиксировать точку, на которую смотрит наблюдатель, либо «привязать» наблюдателя к какому-то объекту, удерживая его на фиксированном расстоянии от этого объекта (эффект слежения). В таком случае ракурс задаётся следующими параметрами:

1. Точка E , определяющая позицию наблюдателя на сцене.
2. Точка C , определяющая центр видимой области.
3. Вектор $\vec{u}$, определяющий направление «вверх» для наблюдателя.

Матрица вида V для камеры от третьего лица формируется по формуле

$$V = \begin{pmatrix} \vec{s}_x & \vec{s}_y & \vec{s}_z & 0 \\ \vec{v}_x & \vec{v}_y & \vec{v}_z & 0 \\ -\vec{f}_x & -\vec{f}_y & -\vec{f}_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 & 0 & 0 & -E_x \\ 0 & 1 & 0 & -E_y \\ 0 & 0 & 1 & -E_z \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad (4.3)$$

где $\vec{f} = \frac{C-E}{|C-E|}$,

$\vec{s} = \vec{f} \times \vec{u}$,

$\vec{v} = \frac{\vec{s}}{|\vec{s}|} \times \vec{f}$.

Часто функцию, формирующую матрицу вида камеры от третьего лица, называют lookAt (от англ. *Look At* – «смотреть на»).

В принципе камеру от первого лица можно реализовать через данную функцию (перевычисляя координаты точек E и C в соответствии с навигационными командами). Такой способ может быть удобен в ряде случаев, однако чаще всего lookAt используется именно для камер от третьего лица.



Задание. Реализовать навигацию по виртуальному миру при помощи камеры от третьего лица. Объект слежения может быть «мнимым», т. е. выражен лишь математически, не имея при этом видимого образа. Сравнить полученную реализацию по удобству и эффективности с предыдущими способами навигации.

4.3. Орбитальная камера

Орбитальная камера используется в ситуациях, когда наблюдатель смотрит на всю сцену целиком, как бы «со стороны». В этом случае он неподвижен, а сцена вращается перед его «глазами» и, возможно, масштабируется в соответствии с командами пользователя. Чаще всего орбитальная камера хорошо подходит для различного рода систем просмотра 3D-моделей, редакторов трёхмерной графики или же в том случае, когда в рамках некоторого виртуального мира необходимо дать пользователю возможность «взять и покрутить в руках» некоторый объект. Для навигации по виртуальному миру орбитальная камера, как правило, непригодна.

Суть орбитальной камеры заключается в том, что она обеспечивает вращение объекта вокруг осей глобальной системы координат. Такая модель вращения может быть достаточно удобно организована с помощью матриц направляющих косинусов или кватерни-

онов. Углы Тайта-Брайана, в свою очередь, оказываются плохо применимы в данном случае, так как основаны на иерархии поворотов.

Идея реализации орбитальной камеры на основе матриц направляющих косинусов состоит в том, чтобы на каждый новый поворот выбирать новую ось в текущей (ранее каким-то образом повернутой) системе координат объекта, совпадающую, однако, с желаемой осью поворота в глобальной системе координат. Несмотря на некоторую громоздкость формулировки требования к повороту, нужная ось может быть найдена очень легко.

Если накапливать поворот в отдельной матрице R , в каждый момент времени она будет описывать разложение базисных векторов новой системы координат в базисе старой. Размерность R , строго говоря, должна быть 3×3 , однако для общности представления и удобства перемножения с другими матрицами имеет смысл хранить её в виде матрицы 4×4 . Чтобы осуществить очередной поворот вокруг оси, заданной вектором $\vec{a}$ в глобальной системе координат, необходимо найти вектор $\vec{a}'$, задающий эту ось в локальной системе координат, переход к которой осуществляется при помощи матрицы R . Векторы $\vec{a}$ и $\vec{a}'$ по определению аффинных преобразований связаны соотношением $\vec{a} = R\vec{a}'$. Следовательно, $\vec{a}' = R^{-1}\vec{a}$.

Так как R – матрица поворота, её обратная матрица совпадает с транспонированной: $R^{-1} = R^T$. Так как желаемая ось поворота, скорее всего, совпадает с одной из координатных осей, для получения вектора $\vec{a}'$ достаточно просто взять соответствующий столбец матрицы R^T , т. е. соответствующую строку матрицы R .

При этом, чтобы избежать аккумуляции погрешности вещественных чисел, после каждого поворота следует производить *локальную перенормировку базиса*, т. е. приводить вектора-столбцы матрицы R к единичной длине.

Обычно поворот орбитальной камеры управляется при помощи движения курсора с нажатой клавишей мыши («драга», англ. *Drag*) или, в случае сенсорных экранов, жеста панарамирования («пана», англ. *Pan*). При этом, однако, удаётся организовать управление вращением только вокруг двух осей, как правило, вертикальной и горизонтальной. Вращение вокруг оси глубины требует дополни-

тельных модификаторов (например, нажатия какой-либо клавиши на клавиатуре, либо использования жеста кручения, англ. *Rotation*).

В листинге 15 приведена концептуальная реализация вращения орбитальной камеры, управляемой при помощи мыши.

Листинг 15. Реализация вращения орбитальной камеры

```
1 void Camera::dragChanged(float dx, float dy)
2 {
3     mat4 r = m_rotation;
4     if (shiftPressed())
5     {
6         r *= mat4::rotation(dx * ROT_SPEED,
7                               r[2], r[6], r[10]);
8     }
9     else
10    {
11        r *= mat4::rotation(dx * ROT_SPEED,
12                              r[1], r[5], r[9]);
13        r *= mat4::rotation(dy * ROT_SPEED,
14                              r[0], r[4], r[8]);
15    }
16    r.normalize();
17    m_rotation = r;
18 }
```

За хранение матрицы R (размерности 4×4) отвечает переменная `m_rotation`. Предполагается, что функция `dragChanged` вызывается всякий раз, когда меняется положение курсора при условии нажатой клавиши мыши (чаще всего левой). При этом функция `shiftPressed` определяет, нажата ли клавиша `Shift`, принятая в данном случае за модификатор, переключающий вращение с осей XY на ось Z . Аргументы `dx` и `dy` хранят величину смещения координат курсора относительно начала драга.

Если нажата клавиша `Shift`, поворот осуществляется вокруг оси Z (в качестве вектора, задающего ось вращения, берётся третья строка матрицы R , строки 6–7 в листинге). В противном случае пово-

рот осуществляется вокруг осей Y (направляющий вектор – вторая строка матрицы R , строки 11–12 в листинге) и X (направляющий вектор – первая строка матрицы R , строки 13–14 в листинге).

Углы поворотов вычисляются на основе смещения курсора. Константа `ROT_SPEED` задаёт линейное преобразование смещения курсора в угол поворота. На практике эта константа может быть подобрана экспериментально. Достаточно естественных результатов можно достигнуть, если принять за правило, что проход курсора через весь экран обеспечивает поворот на 180° :

```
ROT_SPEED = PI / min(screen.width, screen.height);
```

В строке 16 осуществляется локальная нормировка матрицы. Читателю предлагается реализовать эту функцию самостоятельно.

Поворот – центральная составляющая орбитальной камеры. Однако кроме него в её матрицу вида может входить также перенос и масштаб. Перенос обычно используется для того, чтобы расположить объект, просмотр которого обеспечивает камера, в нужном месте на экране, а масштаб используется для организации функции увеличения/уменьшения этого объекта. В этом случае, если обозначить матрицу переноса за T , а матрицу масштаба за S , матрица вида V орбитальной камеры может быть вычислена по формуле

$$V = TSR. \quad (4.4)$$



Задание 1. Реализовать функцию локальной нормировки матрицы размерности 4×4 . Какое максимальное число элементов такой матрицы будет изменено в результате нормировки?

Задание 2. Предложить способ организации орбитальной камеры на основе кватернионов.

5. Загрузка 3D-моделей

Важнейшим компонентом виртуального мира являются трёхмерные объекты, располагаемые в нём вместе с ландшафтом и небом. Эти объекты, как правило, и составляют основное содержание виртуального мира, определяют его назначение и идею. Технически всё, что присутствует на трёхмерной сцене, является объектами (включая рассмотренные ранее ландшафт и небо). Некоторые объекты создаются процедурно, однако большинство загружаются из файлов (будучи предварительно созданными посредством 3D-редакторов и сохранёнными в некотором формате).

Трёхмерные модели состоят из *каркаса* (англ. *Mesh*) и *материала* (англ. *Material*). Каркас описывает геометрические свойства объекта (поверхность) и представляет собой, как правило, полигональную сетку, состоящую из вершин, объединённых в полигоны (треугольники). Кроме того, каркас может хранить дополнительные данные, например, нормали к поверхности в каждой её точке. Материал описывает визуальные (оптические) свойства объекта, часто включает в себя текстуру и тем или иным образом заданный алгоритм вычисления финального внешнего вида (как правило, шейдеры либо параметры для них).

Кроме того, трёхмерная модель может также содержать в себе данные об анимации – изменении формы во времени. Анимация чаще всего используется при моделировании различных персонажей, механизмов и т. п. – объектов, изменение которых не выражается целиком через аффинные преобразования.

В современной компьютерной графике существуют два подхода к анимации 3D-моделей: покадровая (англ. *Keyframe Animation*) и скелетная (англ. *Skeletal Animation*).

Покадровая анимация предполагает хранение нескольких экземпляров каркаса модели, каждый из которых выражает какое-то ключевое состояние формы этой модели («позу»). При проигрывании такой анимации положение вершин интерполируется (линейно либо по какому-то более сложному закону) между соседними ключе-

выми кадрами, чем обеспечивается плавность движений. Такой подход прост в программной реализации, однако требует большого расхода памяти и, что значительно хуже, оказывается очень сложен на этапе создания моделей в 3D-редакторах. Простая покадровая анимация требует создания каждого ключевого кадра в отдельности путём перерасмещения всех вершин каркаса модели, что, во-первых, оказывается очень трудоёмким, а во-вторых, не обеспечивает высокого качества результирующих движений.

В связи с этим на сегодняшний день простая покадровая анимация почти не используется, уступив место скелетной. Скелетная анимация предполагает дополнение каркаса модели т. н. скелетом – совокупностью простых элементов, называемых костями (англ. *Bones*), объединённых в иерархию. Кости существуют только в виде математической модели (не визуализируются на этапе финального рендеринга, хотя, как правило, отображаются во время создания модели в 3D-редакторе) и представляют собой, как правило, отрезки прямых, расположенные «внутри» каркаса. Каждой вершине каркаса назначаются весовые коэффициенты влияния на неё каждой из костей скелета. На этапе визуализации позиция каждой вершины корректируется в соответствии с трансформациями, применёнными к влияющим на неё костям. В результате получается, что скелет играет роль своего рода «опорной конструкции», изменяя которую, можно изменять форму трёхмерного объекта.

Для организации анимации сохраняются ключевые кадры изменения скелета, а не каркаса модели. Скелет устроен значительно проще, чем каркас (как минимум, в нём значительно меньше элементов), в связи с чем редактировать его оказывается намного удобнее, а хранение требует меньше памяти. Кроме того, он представляет собой иерархию, что может накладывать на изменение дополнительные ограничения, помогающие 3D-художнику сделать движения наиболее естественными. Так, например, на сегодняшний день активно используется *прямая и инверсная кинематика*.

Прямая кинематика (англ. *Forward Kinematics*, FK) – это распространение движения от родительских элементов к дочерним. Так, например, если представить себе 3D-модель человека со

скелетом, аналогичным по структуре реальному анатомическому, движение кости плеча (родительского элемента) неизбежно приводит к движению предплечья и кисти (дочерних элементов). В этом случае художнику, чтобы реализовать поднятие руки виртуального персонажа, не нужно двигать все участвующие в этом жесте кости – достаточно сместить только родительскую.

Инверсная кинематика (англ. *Inverse Kinematics*, IK) – это распространение движения от дочерних элементов к родительским. Так, например, если переместить в пространстве кисть руки человека, вслед за ней изменят своё положение и предплечье, и плечо (а если величина перемещения будет достаточно большой, то и другие части, вплоть до всего человека целиком). При этом важную роль играют ограничения подвижности суставов (англ. *Joints*). В этом случае художнику, чтобы реализовать реалистичное махание рукой, достаточно задать траекторию движения кисти, а изменение конфигурации руки будет вычислено автоматически. Такой подход оказывается наиболее интуитивным, так как хорошо согласуется с человеческим опытом (как правило, для совершения тех или иных действий человек продумывает траекторию именно самого «младшего» элемента в иерархии своего скелета: например, чтобы нажать на клавишу, вы, скорее всего, концентрируете внимание на кончике пальца и совершенно не задумываетесь о том, на какие углы при этом должны поворачиваться локтевой, плечевой суставы и запястье).

Существует огромное множество различных форматов хранения 3D-моделей (как бинарных, так и текстовых). Некоторые из них позволяют сохранять только каркас (например, форматы STL и PLY), другие предусматривают хранение материалов (например, OBJ), покадровой (например, MD2) и скелетной анимации (например, MD5). Также имеются форматы комплексного хранения трёхмерных сцен, ориентированные на полноценный обмен данными между различными системами обработки 3D-графики (например, COLLADA).

Наиболее простыми из популярных на сегодняшний день форматов являются STL, PLY и OBJ. Загрузчик для них может быть без

труда реализован вручную. Для загрузки 3D-моделей в более сложных форматах, как правило, используются сторонние библиотеки.

Загрузка модели включает в себя создание вершинных и индексных буферов, хранящих данные каркаса (включая текстурные координаты вершин, координаты нормалей к поверхности и т. п.), загрузку текстур и настройку шейдеров (в которых, в частности, реализуется модель освещения для соответствующего объекта). Реализации загрузчиков для популярных форматов (а также специализированные библиотеки) без труда могут быть найдены в сети Интернет.



Задание. Наполнить виртуальный мир объектами (например, растительностью), загруженными из внешних файлов. При выводе объектов использовать текстурирование и какую-либо модель освещения, например, модель Фонга.

Подсказка. Наиболее удобные форматы, с которых можно начать знакомство с процессом загрузки 3D-моделей, – PLY и OBJ. Они просты с точки зрения спецификации, однако при этом содержат достаточно информации (в частности, поддерживают хранение текстурных координат и нормалей). Модели в формате OBJ без труда можно найти в сети Интернет, а конвертация в формат PLY поддерживается свободным и бесплатно распространяемым 3D-редактором Blender.

6. Рельефное текстурирование

Как правило, реалистичность внешнего вида объекта напрямую связана с его детализированностью. В реальном мире объекты, на первый взгляд обладающие простой формой (например, столы и стулья), при ближайшем рассмотрении являют наблюдателю огромное множество мелких деталей: технические и декоративные элементы конструкции, разнообразные дефекты, такие как трещины, сколы, царапины и прочие неровности поверхности, и т. п. Объекты «идеальной» формы (лишённые дефектов и других характерных мелочей), будучи встроенными в виртуальный мир, выдают его синтетическую природу, разрушая у пользователя общую иллюзию реалистичности происходящего. В связи с этим на сегодняшний день гонка за детализацией – едва ли не основной двигатель развития компьютерной графики (как на программном, так и на аппаратном уровнях).

Детализация объекта часто сводится к приданию его поверхности некоторого *рельефа* (англ. *Relief*) – дополнительных искривлений относительно базовой формы. Существует много различных подходов к созданию и контролю рельефа, например, процедурная генерация (когда рельеф является изменением поверхности некоторой функцией) или «скульптурирование» (англ. *Sculpting*, когда специальные инструменты в составе 3D-редакторов позволяют виртуально «вылепить» неровности на поверхности).

Очень популярным и гибким, а также относительно простым и эффективным в реализации подходом является также *рельефное текстурирование* (англ. *Relief Mapping*) – метод моделирования рельефа поверхности объекта на основе текстуры. Идея рельефного текстурирования заключается в том, что данные о рельефе поверхности объекта некоторым образом сохраняются в структуре, которая затем обрабатывается в графическом приложении при помощи текстурных механизмов. Проще говоря, для хранения сведений о рельефе используется текстура.

Рельефное текстурирование подразделяется на два вида: *наложение карты смещений* (англ. *Displacement mapping*) и *имитация рельефа* (англ. *Relief Faking*).

Карта смещений описывает изменение положений вершин поверхности с целью придания рельефа на уровне геометрической формы. Однако даже современное оборудование далеко не всегда способно обеспечить нужный уровень детализации геометрических структур. Особенно это касается передачи т. н. *микрорельефа* (англ. *Microrelief*) – неровностей на поверхности, размер которых значительно меньше размера объекта. Если передавать все эти неровности при помощи изменений на уровне каркаса 3D-модели, необходимо использовать для аппроксимации нужной поверхности полигоны очень малой площади, что соответственно резко увеличивает число требуемых для представления объекта вершин и сильно нагружает систему визуализации.

Для решения этой проблемы используется комплекс приёмов по имитации микрорельефа на уровне материала (визуальных свойств) объекта. Простейший способ такой имитации – наложение обычной текстуры. Однако в условиях наличия некоторого освещения, порождающего игру света и тени, эффект такой имитации сразу пропадает: текстура кажется лишь плоским узором, нанесённым на поверхность объекта (чем она, в сущности, и является), а не рельефом, так как тени и блики рассчитываются относительно гладкой поверхности. Для достижения иллюзии рельефности осуществляется воздействие текстуры на нормали: изменение поля нормалей поверхности так, чтобы поведение теней и бликов создавало иллюзию присутствия неровностей. Данные для такого изменения извлекаются из текстуры.

На рис. 14 продемонстрированы результаты использования трёх разных подходов: обычное наложение текстуры, имитация рельефа и наложение карты смещений.

На границе объекта, изображённого на рис. 14б, отчётливо видно, что изображаемый рельеф фактически не является объёмным (в отличие от рис. 14в), однако объект в целом производит впечатление достаточно высокодетализированного.

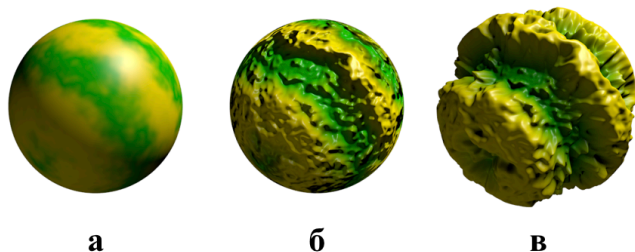


Рис. 14. Пример визуализации сферы с наложением текстуры (а), имитацией рельефа (б) и наложением карты смещений (в)

Имитация рельефа может быть осуществлена несколькими разными способами, основными из которых являются:

1. *Наложение карты неровностей* (англ. *Bump Mapping*) – наиболее примитивный способ рельефного текстурирования, пригодный для создания очень неглубоких неровностей.
2. *Наложение карты нормалей* (англ. *Normal Mapping*) – наиболее популярный способ рельефного текстурирования, пригодный для создания достаточно глубоких и разнообразных неровностей, допускающий при этом простую и эффективную реализацию.
3. *Наложение карты параллакса* (англ. *Parallax Mapping*) – более сложный и в то же время более качественный способ рельефного текстурирования, пригодный для создания глубоких неровностей.
4. *Наложение карты параллакса с учётом заслонений* (англ. *Parallax Occlusion Mapping*) – наиболее вычислительно-сложный способ рельефного текстурирования, обеспечивающий наилучшее качество и позволяющий создать очень глубокий и разнообразный рельеф.

Рассмотрим подробнее наложение карты нормалей как способ, сочетающий в себе достаточную гибкость, универсальность и простоту реализации. Он является очень эффективным с точки зрения итоговой производительности, так как не требует уплотнения сетки каркаса 3D-модели (число полигонов не увеличивается), а рельеф создаётся, как правило, на этапе преобразования фрагментов (во фраг-

ментном шейдере, как часть кода, реализующего модель освещения). Тем самым экономятся ресурсы вычислительной системы, но при этом увеличивается визуальное качество результата.

Основная суть наложения карты нормалей состоит в том, что нормаль в заданном фрагменте (необходимая для вычисления освещённости этого фрагмента в соответствии с той или иной моделью освещения) не вычисляется билинейной интерполяцией нормалей соответствующих вершин полигона (как в методе Фонга), а извлекается из специальной текстуры (карты нормалей) по текстурным координатам этого фрагмента. Результирующее поле нормалей полигона схематично изображено на рис. 15.

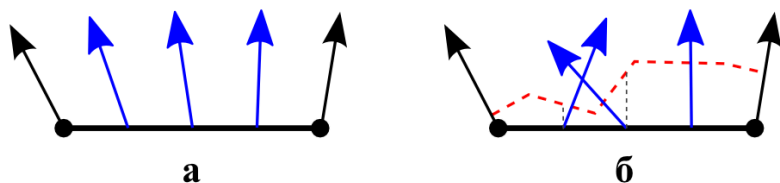


Рис. 15. Схематичное изображение полей нормалей полигона при вычислении по Фонгу (а) и наложении карты нормалей (б)

Чёрные стрелки обозначают нормали, заданные в вершинах объекта, синие – нормали, вычисленные для его фрагментов. Пунктиром на рис. 15б обозначен рельеф, имитируемый картой нормалей. Получается, что поле нормалей визуализируемого объекта подменяется вычисленным на базе специальной текстуры, и с «точки зрения» моделирования освещения объект приобретает рельеф, отсутствующий на уровне его полигональной сетки.

Карта нормалей обычно представляет собой обычную текстуру, значения цветовых компонентов которой интерпретируются как координаты нормалей. Однако такая интерпретация требует преобразования, так как значение цвета при выборке из текстуры меняется в отрезке $[0; 1]$, а значение координат нормированного вектора – в отрезке $[-1; 1]$. Если обозначить цвет за $\vec{c}$ (работая с ним, как с трёхмерным вектором), преобразование в координаты нормали имеет вид

$$\vec{n} = 2\vec{c} - \vec{e}, \quad (6.1)$$

где $\vec{e} = \{1; 1; 1\}$.

Источником карты нормалей для 3D-модели может быть как некоторый сгенерированный узор, так и более высокодетализированная версия этой модели. Процесс сохранения нормалей детализированной 3D-модели в текстуру с целью использования их при визуализации более простой модели называется *запеканием нормалей* (англ. *Normal Baking*). Большинство современных 3D-редакторов поддерживают такую функцию. На рис. 16 продемонстрировано использование запекания нормалей для упрощения полигональной сетки модели без уменьшения визуального качества.

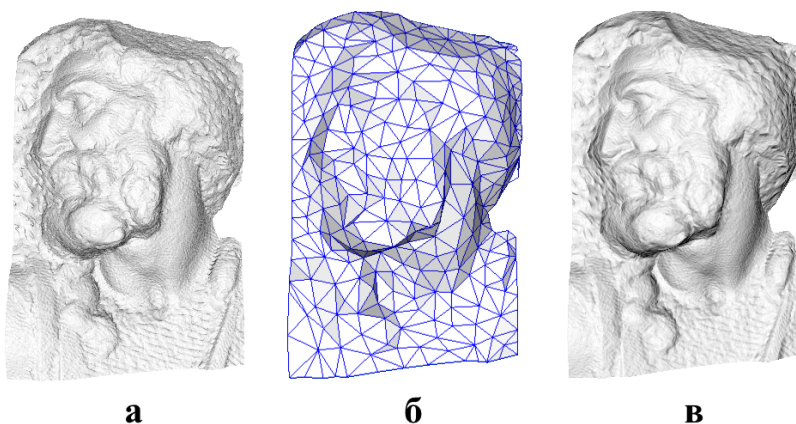


Рис. 16. Использование запекания нормалей: исходная 3D-модель из 4 миллионов полигонов (а), упрощённая модель из 500 полигонов (б), упрощённая модель с наложением карты нормалей (в)

(автор изображения Paolo Cignoni, CC BY-SA 1.0,
https://en.wikipedia.org/wiki/Normal_mapping#/media/File:Normal_map_example.png)

Как можно видеть, рассматриваемая технология может обеспечивать упрощение 3D-модели на несколько порядков, т. е. является очень действенным методом оптимизации графической сцены.

Существуют два способа задания нормалей посредством текстуры (с учётом соотношения (6.1)): в *пространстве объекта* (англ. *Object Space*) и в *касательном пространстве* (англ. *Tangent Space*). На рис. 17 показан 3D-объект, визуализируемый с применением рельефного текстурирования, и два варианта карты нормалей для него.

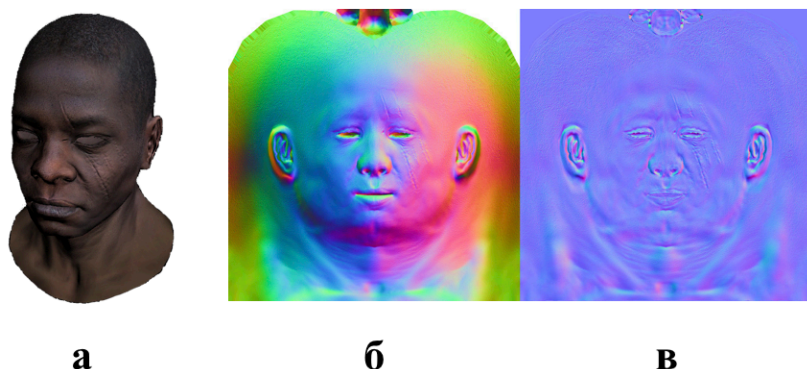


Рис. 17. Трёхмерный объект (а) и варианты карты нормалей для него: нормали заданы в пространстве объекта (б) и в касательном пространстве (в)

Карта нормалей, заданных в пространстве объекта, отличается большим цветовым разнообразием, обусловленным разнообразием направлений нормалей. Так, например, левая щека персонажа (находящаяся справа на текстуре) имеет красноватый оттенок: нормали к её поверхности близки по направлению к оси X (которая проходит слева направо), следовательно, значение первого компонента, соответствующего красной составляющей цвета на текстуре, оказывается больше остальных. Правая щека, наоборот, имеет синий оттенок, так как красный компонент в соответствии с формулой (6.1) оказывается близок к нулю (тогда как нормали к поверхности имеют отрицательное значение координаты x).

Такой способ представления карты нормалей достаточно удобен, так как координаты векторов нормалей могут быть напрямую извлечены из карты без дополнительных вычислений (а затем преобразованы матрицей нормалей, как было описано в разделе 1.3.2). При

этом «геометрические» нормали 3D-модели («настоящие» перпендикуляры к её поверхности) оказываются излишними и не используются при вычислении освещения.

Однако вместе с указанным достоинством у данного способа есть и серьёзный недостаток: отсутствие инвариантности карты нормалей относительно ориентации фрагментов поверхности. Это означает, что каждая отдельно взятая часть текстуры может быть корректно использована только для соответствующей части поверхности и ни для какой другой, т. е. невозможно переиспользование текстурных областей для разных зон на 3D-модели. Например, если имеется 3D-модель доспехов рыцаря, на обоих наплечных щитках которых должна быть изображена гравировка герба, соответствующий узор придётся повторить на текстуре дважды (так как нормали на правом и левом наплечных щитках разнонаправлены). Это, в свою очередь, приводит к дополнительному расходу памяти. Соответственно невозможным оказывается также использование текстур-шаблонов, таких как изображения царапин, нерегулярного микро рельефа (покрывающего большие площади с использованием описанного ранее режима повторения) и т. п. Кроме того, если модель изменяет форму с течением времени (анимирована), направление нормалей также будет меняться, и придётся хранить отдельные текстуры для каждого ключевого кадра анимации, а затем интерполировать их.

Этот недостаток нивелируется заданием нормалей в касательном пространстве (которое также иногда называют *тангенциальным*). Касательное пространство для поверхности 3D-модели в заданной точке — это совокупность касательных векторов к поверхности в этой точке с введённой структурой векторного пространства. В общем случае в каждой точке поверхности касательное пространство своё. Множество касательных пространств для всех точек поверхности называется *касательным расслоением* (англ. *Tangent Bundle*) этой поверхности, а каждое конкретное касательное пространство — *слоем* (англ. *Fiber*) этого расслоения. Следует отметить, что размерность касательного пространства для трёхмерной поверхности также равна 3 (более подробно с теорией касательных пространств можно познакомиться в книге [9]).

Идея формирования карты нормалей в касательном пространстве заключается в том, что в каждом текселе нормаль хранится в таком виде, как если бы касательная плоскость к поверхности в этой точке совпадала с плоскостью текстуры. Можно сказать, текстура в этом случае хранит не сами нормали, а их отклонения, что обеспечивает инвариантность относительно ориентации поверхности.

Обычно для представления нормалей принято вводить в касательном пространстве ортонормированный базис, ось Z которого совпадает по направлению с «геометрической» нормалью к поверхности в данной точке. Поскольку нормали при рельефном текстурировании, как правило, отклоняются от своих первоначальных направлений лишь незначительно, в цветовом представлении текстуры преобладает синий компонент (соответствующий координате z).

Для вычисления освещённости точки необходимо, чтобы все задействованные в модели освещения геометрические элементы (нормаль, вектор падения света, вектор взгляда и т. п.) были заданы в одной системе координат (не важно, в какой именно, важно – чтобы в одной и той же для всех составляющих). В связи с этим необходимо построить модуль перехода между пространством сцены и касательным пространством объекта.

Перейти из пространства сцены в пространство объекта можно при помощи матрицы преобразования объекта (и вычисленной на её основе матрицы нормалей). Таким образом, вышеуказанная задача может быть сведена к поиску модуля перехода от пространства объекта к его касательному пространству. Этот модуль основывается на аффинных преобразованиях, а точнее – на преобразовании поворота, так как касательное пространство отличается от пространства объекта лишь ориентацией. В качестве базиса касательного пространства удобно использовать *трёхгранник Френе-Серре* (англ. *Frenet–Serret Frame*) – совокупность *касательной* (англ. *Tangent*, берётся за ось X , обозначается $\vec{t}$), *бинормали* (англ. *Binormal*, берётся за ось Y , обозначается $\vec{b}$) и *нормали* (англ. *Normal*, берётся за ось Z , обозначается $\vec{n}$). Бинормаль кривой – это вектор, перпендикулярный плоскости, образованной векторами нормали и касательной. Кривой в данном случае выступает *некоторое* сечение поверхности.

Использование трёхгранника Френе-Серре и задание конкретных соответствий касательной, бинормали и нормали осям координат касательного пространства является стандартным соглашением (конвенционализмом) и обусловлено удобством сопутствующих вычислений.

Предполагается, что нормаль $\vec{n}$ к поверхности в заданной точке определяется однозначно (с математической точки зрения она единственна, хотя на практике, при отсутствии аналитического задания поверхности, может быть вычислена по-разному). Нормаль выражается в пространстве объекта (её координаты заданы в базисе системы координат объекта).

Бинормалью $\vec{b}$ и касательной $\vec{t}$, строго говоря, могут быть любые два ортогональных единичных вектора, лежащих в касательной плоскости (которая однозначно определяется через нормаль). Однако преобразование цвета, извлекаемого из текстуры, в координаты нормали должно быть *согласованным* для всех текселей, в противном случае результат рельефного текстурирования окажется некорректным. Следовательно, базис касательного пространства необходимо всегда выбирать по одному и тому же принципу (и этот принцип должен быть одинаковым как при построении карты нормалей, так и при визуализации объекта).

Как отмечалось выше, карта нормалей строится исходя из предположения, что она лежит в касательной плоскости. С точки зрения простоты вычислений в этом случае удобно положить касательную и бинормаль совпавшими по направлению с осями U и V соответственно, т. е. с базисом текстурного пространства карты нормалей. Необходимо найти разложение базисных векторов текстурного пространства в пространстве объекта.

Предполагается, что у каждой точки P поверхности заданы пространственные координаты $\{x, y, z\}$ в пространстве объекта (в противном случае визуализация поверхности невозможна) и текстурные координаты $\{u, v\}$ в пространстве текстуры (в противном случае текстурирование – как рельефное, так и обычное – неосуществимо). На рис. 18 показана проекция полигона, представленного точками P_1, P_2, P_3 , на текстуру.

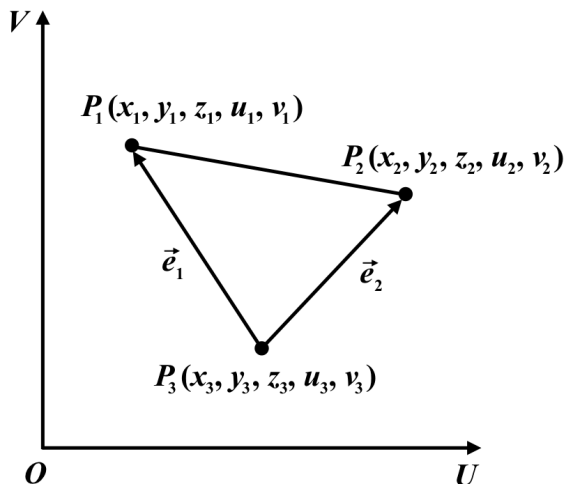


Рис. 18. Проекция полигона трёхмерной поверхности в текстурное пространство

Векторы $\vec{e}_1$ и $\vec{e}_2$ выражаются в пространстве текстуры следующим образом:

$$\begin{aligned}\vec{e}_1 &= (u_1 - u_3)\vec{u} + (v_1 - v_3)\vec{v} \\ \vec{e}_2 &= (u_2 - u_3)\vec{u} + (v_2 - v_3)\vec{v},\end{aligned}\tag{6.2}$$

где $\vec{u}, \vec{v}$ – орты пространства текстуры.

С другой стороны, в пространстве объекта эти векторы выражаются так:

$$\begin{aligned}\vec{e}_1 &= (x_1 - x_3)\vec{i} + (y_1 - y_3)\vec{j} + (z_1 - z_3)\vec{k} \\ \vec{e}_2 &= (x_2 - x_3)\vec{i} + (y_2 - y_3)\vec{j} + (z_2 - z_3)\vec{k},\end{aligned}\tag{6.3}$$

где $\vec{i}, \vec{j}, \vec{k}$ – орты пространства объекта.

Из формул (6.2) и (6.3) нетрудно выразить векторы $\vec{u}$ и $\vec{v}$ через $\vec{i}, \vec{j}, \vec{k}$, т. е. найти разложение базиса текстурного пространства в пространстве объекта. Далее, векторы $\vec{t}$ и $\vec{b}$ полагаются равными $\vec{u}$ и $\vec{v}$ соответственно.

Стоит отметить, что на практике достаточно найти лишь один из векторов $\vec{t}$ и $\vec{b}$, так как имеет место соотношение

$$\vec{b} = \vec{n} \times \vec{t}. \quad (6.4)$$

Из полученных векторов $\vec{t}$, $\vec{b}$, $\vec{n}$, как из столбцов, составляется матрица, называемая TBN , преобразующая точки из касательного пространства в пространство объекта. Для связи касательного пространства с пространством сцены эта матрица слева должна быть умножена на матрицу нормалей N . Результат носит название $NTBN$. В общем случае для каждой вершины объекта $NTBN$ принимает свои значения элементов (так как координаты нормали, би-нормали и касательной различаются).

Для вычисления освещения все векторы, участвующие в этом процессе, необходимо привести к разложению в едином базисе. Матрица $NTBN$ допускает билинейную интерполяцию и, будучи сформированной в вершинном шейдере, может быть передана во фрагментный. Однако с целью упрощения кода фрагментного шейдера и повышения производительности визуализации матричных операций в нём рекомендуется избегать. Для этого в качестве «целевого» пространства (в базисе которого раскладываются все векторы) обычно используют касательное: в таком случае нормаль, извлечённую из текстуры, не нужно дополнительно преобразовывать.

Для перехода от пространства сцены в касательное пространство используется матрица $NTBN^{-1}$. Часто на практике матрица $NTBN$ является матрицей поворота (без «примеси» масштабирования). В этом случае $NTBN^{-1} = NTBN^T$.

Иногда касательные векторы сохраняются в файле модели, будучи посчитанными в 3D-редакторе на этапе создания. Однако чаще всего их приходится вычислять на стороне графического приложения. Код вычисления касательных векторов приведён в листинге 16.

За `vertices` обозначим линейный массив, содержащий все атрибуты всех вершин объекта. За `indexArrayLength` обозначим длину массива индексов.

Листинг 16. Вычисление касательных векторов для точек поверхности
3D-модели

```
1  for (int i = 0; i < indexArrayLength; i += 3)
2  {
3      vec3 eo1(vertices[iX(1)] - vertices[iX(0)],
4              vertices[iY(1)] - vertices[iY(0)],
5              vertices[iZ(1)] - vertices[iZ(0)]);
6      vec3 eo2(vertices[iX(2)] - vertices[iX(0)],
7              vertices[iY(2)] - vertices[iY(0)],
8              vertices[iZ(2)] - vertices[iZ(0)]);
9
10     vec2 et1(vertices[iU(1)] - vertices[iU(0)],
11             vertices[iV(1)] - vertices[iV(0)]);
12     vec2 et2(vertices[iU(2)] - vertices[iU(0)],
13             vertices[iV(2)] - vertices[iV(0)]);
14
15     float f = 1.0f / (et1.x*et2.y - et2.x*et1.y);
16
17     vec3 tangent(f * (et2.y*eo1.x - et1.y*eo2.x),
18                 f * (et2.y*eo1.y - et1.y*eo2.y),
19                 f * (et2.y*eo1.z - et1.y*eo2.z));
20     tangent.normalize();
21
22     for (int j = 0; j < 3; ++j)
23     {
24         vec3 t(vertices[iTX(j)],
25               vertices[iTY(j)],
26               vertices[iTZ(j)]);
27         t += tangent;
28         t.normalize();
29         vertices[iTX(j)] = t.x;
30         vertices[iTY(j)] = t.y;
31         vertices[iTZ(j)] = t.z;
32     }
33 }
```

Приведённый в листинге цикл совершает обход всех треугольников модели (строка 1).

В целях компактности кода определим макросы iX , iY , iZ , iU , iV , iTX , iTY , iTZ , которые обеспечивают вычисление индексов соответствующих атрибутов по заданному номеру вершины:

```
#define iX(nmb) (indices[i + (nmb)] * vComp + 0)
#define iY(nmb) (indices[i + (nmb)] * vComp + 1)
#define iZ(nmb) (indices[i + (nmb)] * vComp + 2)
```

Остальные макросы определяются аналогично. Здесь $indices$ – массив индексов 3D-модели, $vComp$ – константа, задающая число атрибутов у вершины, i – внешняя переменная-итератор, используемая в цикле.

В строках 3–13 листинга 16 вычисляются разложения векторов $\vec{e}_1$ и $\vec{e}_2$. Затем в строках 15–20 вычисляется и нормируется вектор $\vec{t}$. Вектор $\vec{b}$ в данном примере будет определён позднее (в вершинном шейдере), чтобы сэкономить память и не хранить его в атрибутах вершины. В строках 22–32 происходит аккумуляция касательной в вершинах обрабатываемого треугольника. Алгоритм, реализованный для этого, аналогичен алгоритму сглаживающих групп для вычисления нормалей (который был описан в разделе 1.3.2). Таким образом обеспечивается интерполяция (сглаживание) касательных, что увеличивает визуальное качество результатов рендеринга.

В результате исполнения приведённого цикла во всех вершинах модели сохраняются координаты касательных векторов (в виде атрибутов, индексы которых в массиве вершин определяют макросы iTX , iTY , iTZ).

Приведённый метод является универсальным (как и алгоритм сглаживающих групп для вычисления нормалей), т. е. может быть использован для полигональных 3D-моделей любого типа.

Для осуществления наложения карты нормалей в вершинном шейдере производится подготовительная работа по переносу всех участвующих в вычислении освещения геометрических элементов в касательное пространство. Пример вершинного шейдера, предназначенного для вывода объекта с моделью освещения Фогна, приведён в листинге 17.

Листинг 17. Код вершинного шейдера для наложения карты нормалей

```
1  #version 330
2
3  uniform mat4 u_mvp;
4  uniform mat4 u_mv;
5  uniform mat3 u_n;
6  uniform vec3 u_light;
7
8  in vec3 a_position;
9  in vec3 a_normal;
10 in vec2 a_texCoord;
11 in vec3 a_tangent;
12
13 out vec3 v_ray;
14 out vec3 v_eye;
15 out vec2 v_texCoord;
16
17 void main()
18 {
19     vec4 pos = vec4(a_position, 1.0);
20     vec3 binormal = normalize(cross(a_normal,
21                                     a_tangent));
22     mat3 intbn = inverse(u_n * mat3(a_tangent,
23                                     binormal,
24                                     a_normal));
25     vec3 vPos = intbn * (u_mv * pos).xyz;
26     v_ray = normalize(intbn *
27                       vec4(u_light, 1.0).xyz -
28                       vPos);
29     v_eye = normalize(-vPos);
30     v_texCoord = a_texCoord;
31     gl_Position = u_mvp * pos;
32 }
```

Юниформ-переменные, объявленные в строках 3–6, имеют следующее назначение: `u_mvp` – результат произведения матриц проекции и трансформации, `u_mv` – матрица трансформации, `u_n` – матрица нормалей (соответствующая матрице `u_mv`), `u_light` – радиус-вектор, задающий позицию точечного источника света.

Переменные-атрибуты, объявленные в строках 8–11, имеют следующее назначение: `a_position` – координаты вершины, `a_normal` – координаты нормали, `a_texCoord` – текстурные координаты, `a_tangent` – координаты касательной.

Выходными переменными являются: `v_ray` – вектор, обратный вектору падения света, `v_eye` – вектор взгляда, `v_texCoord` – текстурные координаты.

Основные действия шейдера сосредоточены в строках 20–29, где путём векторного произведения нормали `a_normal` и касательной `a_tangent` вычисляется бинормаль `binormal`, затем строится матрица $NTBN^{-1}$ (`intbn`), и с её помощью координаты позиции точки на поверхности `pos` и координаты источника света `u_light` отображаются в касательное пространство, чтобы на их основе затем посчитать вектор от вершины к источнику света `v_ray` и вектор взгляда `v_eye`.

Следует отметить, что если матрица нормалей `u_n` содержит только поворот, функция `inverse` в строке 22 может быть заменена на `transpose` (что немного увеличит скорость исполнения шейдера).



Задание 1. Реализовать фрагментный шейдер для наложения карты нормалей.

Задание 2. Реализовать рельефное текстурирование путём наложения карты нормалей на один или несколько объектов, размещённых в созданном вир-

туальном мире.

Подсказка. Карту нормалей с некоторым рельефом без труда можно найти в сети Интернет.

Заключение

В данном учебном пособии рассмотрены базовые приёмы и технологии создания трёхмерного виртуального мира. Описаны основные подходы к моделированию ландшафта, неба и освещения, разобраны различные техники наложения текстур (включая рельефное текстурирование), затронуты вопросы загрузки 3D-объектов, приведены основные способы организации интерактивной навигации в трёхмерном пространстве.

Студент, планомерно выполнявший все сформулированные в пособии задания, должен получить в результате программу, которая строит виртуальный мир на основе открытого пространства. Для закрепления и углубления полученных знаний и навыков, ему предлагается создать на базе этого мира полноценное приложение виртуальной реальности (или, как минимум, приложение компьютерной графики): игру с некоторым сюжетом или симулятор.

За рамками пособия остались вопросы организации полного погружения, к которым относятся стереоскопическая визуализация, управление при помощи пространственных жестов и т. п., так как оно ориентировано на начальный этап освоения технологий виртуальной реальности. Рекомендуется продолжить изучение этих технологий самостоятельно, обратив внимание на такие технические средства (и руководства разработчика к ним), как шлемы виртуальной реальности Oculus Rift, Google Cardboard или их аналоги.

Список литературы

1. Burden D. Virtual Reality: for Training, Learning, Collaboration and Visualisation. Daden, 2014. [Электронный ресурс]. URL: <http://www.daden.co.uk/downloads/Virtual%20Reality%20v1.pdf> (дата обращения: 22.10.2017).
2. Баяковский Ю. М., Игнатенко А. В. Начальный курс OpenGL. М.: Планета Знаний, 2007. 221 с.
3. The OpenGL Graphics System: A Specification 4.2 // Khronos Group. 591 p. [Электронный ресурс]. URL: <https://www.khronos.org/registry/OpenGL/specs/gl/glspec42.core.pdf> (дата обращения: 22.10.2017).
4. Anisotropic Filtering // Khronos Group. [Электронный ресурс]. URL: https://www.khronos.org/opengl/wiki/Sampler_Object#Anisotropic_filtering (дата обращения: 22.10.2017).
5. Extension EXT_texture_filter_anisotropic // Khronos Group. [Электронный ресурс]. URL: https://www.khronos.org/registry/OpenGL/extensions/EXT/EXT_texture_filter_anisotropic.txt (дата обращения: 22.10.2017).
6. Phong B. T. Illumination for computer generated pictures // Communications of ACM 18. 1975. No. 6. P. 311–317.
7. Рябинин К. В. Вычислительная геометрия и алгоритмы компьютерной графики. Работа с 3D-графикой средствами OpenGL: учеб. пособие / Перм. гос. нац. исслед. ун-т. Пермь, 2017. 100 с.
8. The Normal Matrix // Lighthouse3d. [Электронный ресурс]. URL: <http://www.lighthouse3d.com/tutorials/glsl-12-tutorial/the-normal-matrix/> (дата обращения: 22.10.2017).
9. Позняк Э. Г., Шикин Е. В. Дифференциальная геометрия: первое знакомство. М.: Изд-во МГУ, 1990. 384 с.

Учебное издание

Рябинин Константин Валентинович

**ВИРТУАЛЬНАЯ РЕАЛЬНОСТЬ
И МУЛЬТИМЕДИА
ПОСТРОЕНИЕ ВИРТУАЛЬНОГО МИРА
СРЕДСТВАМИ OPENGL**

Учебное пособие

Редактор *Л. Г. Подорова*
Компьютерная вёрстка: *К. В. Рябинин*

Подписано в печать 19.02.2018. Формат 60х84/16.
Усл. печ. л. 5,81. Тираж 100 экз. Заказ_____

Издательский центр
Пермского государственного
национального исследовательского университета.
614990, г. Пермь, ул. Букирева, 15

Типография ПГНИУ.
614990, г. Пермь, ул. Букирева, 15